Chapter 2

Search Structures

References:

1. R. Sedgewick and K. Wayne, Algorithms (4th Ed.), Addison-Wesley.
2. E. Horowitz, S. Sahni, S. Anderson-Freed, Fundamentals of Data Structures in C, Silicon Press, 2nd Edition.

TABLE OF CONTENTS

- Symbol Table
- Binary Search Trees
- Optimal Binary Search Trees
- AVL Trees
- Two-Three Trees (2-3 Trees)
- Two-Three-Four Trees (2-3-4 Trees)
- Red-Black Trees

1. Symbol Table

- Symbol Table이란?
- Symbol Table의 API
- 구현 1: 연결 리스트를 이용한 순차 검색
- 구현 2: 정렬된 배열을 이용한 이진 검색
- Ordered Symbol Table

1.1 Symbol Table이란?

- (키, 값) 쌍의 모임
  - 특정 키와 그 키에 해당되는 값의 쌍을 삽입
  - 키가 주어질 때, 관련된 값을 검색

- 예: DNS 검색
  - (새로운 domain 이름, IP 주소) 쌍을 삽입
  - Domain 이름이 주어지면, IP 주소를 반환
    - 예: DNS Lookup

Domain name	IP adress
www.yu.ac.kr	165.229.11.77
cse.yu.ac.kr	165.229.11.154
www.google.com	216.58.218.164
www.amazon.com	54.239.17.6

Symbol Table의 응용 분야

응용 분야	검색의 목적	키	값
사전	단어의 뜻	단어	뜻
책의 색인	관련된 페이지	키워드	페이지 번호 리스트
P2P 음악 서버	다운받을 음악	노래 제목	컴퓨터 ID
은행	트랜잭션 처리	계좌 번호	잔고와 거래 내역
웹 검색	웹 페이지 검색	키워드	URL 리스트
컴파일러	변수의 속성	변수 이름	타입과 값
라우팅 테이블	패킷 전송	목적지	목적지로 가기 위한 경로
DNS	IP 주소 검색	도메인 이름	IP 주소
전화번호부	전화 번호 검색	이름	전화 번호
유전자 공학	DNA 위치 검색	DNA 문자열	알려진 위치
파일 시스템	디스크에서 파일 검색	파일 이름	디스크에서 블록 위치

1.2 Symbol Table의 API

- Associative array의 개념: 각 키에 하나의 값만 할당

public class ST<Key, Value>

ST()	새로운 테이블 생성	
void put(Key k, Value v)	(k, v) 쌍을 테이블에 삽입 ←	a[k] = v
Value get(Key k)	k에 해당되는 값을 반환 ←	a[k]
boolean contains(Key k)	k에 해당되는 값이 있으면 true	
void delete(Key k)	테이블에서 k의 쌍을 삭제	
boolean isEmpty()	테이블이 비었으면 true	
int size()	(k, v) 쌍의 수를 반환	
Iterator<Key> keys()	테이블에 저장된 모든 키들	

고려 사항

- Generics 지원
  - 단, 키는 Comparable 인터페이스를 구현할 것
- 중복 키는 허용하지 않음
  - 테이블에 <k1, v>가 있으며 <k1, v'>가 입력되면 k1의 값을 v'로 변경
- 키와 값은 null이 될 수 없음
  - get(k1): k1이 없을 경우, null을 반환
- 테이블에서 <k1, v>를 삭제하고자 할 경우
  - 방법 1: 실제로 k1을 검색하여 삭제
  - 방법 2: put(k1, null)
- 아래 연산들은 one-line implementation
  - void delete(Key k) → put(k, null);
  - boolean contains(Key k) → return get(k) != null;
  - boolean isEmpty() → return size() == 0;

Sample Clients

- Symbol table을 위한 다양한 알고리즘들을 구현
- 각 알고리즘의 성능을 평가하기 위한 두 가지 샘플 프로그램
  - Test Client
    - 프로그램이 정상적으로 동작하는지 확인
  - Performance Client (FrequencyCounter)
    - 단어의 빈도수를 계산한 후 빈도수가 가장 많은 단어를 출력
  - 세 가지 데이터 파일 제공

	tinyTale.txt		tale.txt		leipzig1M.txt	
	words	distinct	words	distinct	words	distinct
all words	60	20	135,635	10,679	21,191,455	534,580
at least 8 letters	3	3	14,350	5,737	4,239,597	299,593
at least 10 letters	2	2	4,582	2,260	1,610,829	165,555

Characteristics of larger test input streams

TestClient.java

```
public class TestClient {
    public static void main(String[] args) {
        ST<String, Integer> st = new ST<String, Integer>();
        File file;
        final JFileChooser fc = new JFileChooser();      // 파일 선택기를 사용
        if (fc.showOpenDialog(null) == JFileChooser.APPROVE_OPTION)     file = fc.getSelectedFile();
        else  {
                JOptionPane.showMessageDialog(null, "파일을 선택하세요.", "오류", JOptionPane.ERROR_MESSAGE);
                return;
        }

        Scanner sc = null;    // file에 있는 단어들을 키로 해서 테이블에 차례대로 저장
        try {                 // 이후, 테이블에 저장된 모든 (키,값)의 쌍들을 출력
            sc = new Scanner(file);
            for (int i = 0; sc.hasNext(); i++)  { String key = sc.next();      st.put(key, i); }
            for (String s : st.keys())     System.out.println(s + " " + st.get(s));
        } catch (FileNotFoundException e) { e.printStackTrace(); }
        if (sc != null)       sc.close();
}  }
```

FerquencyCounter.java

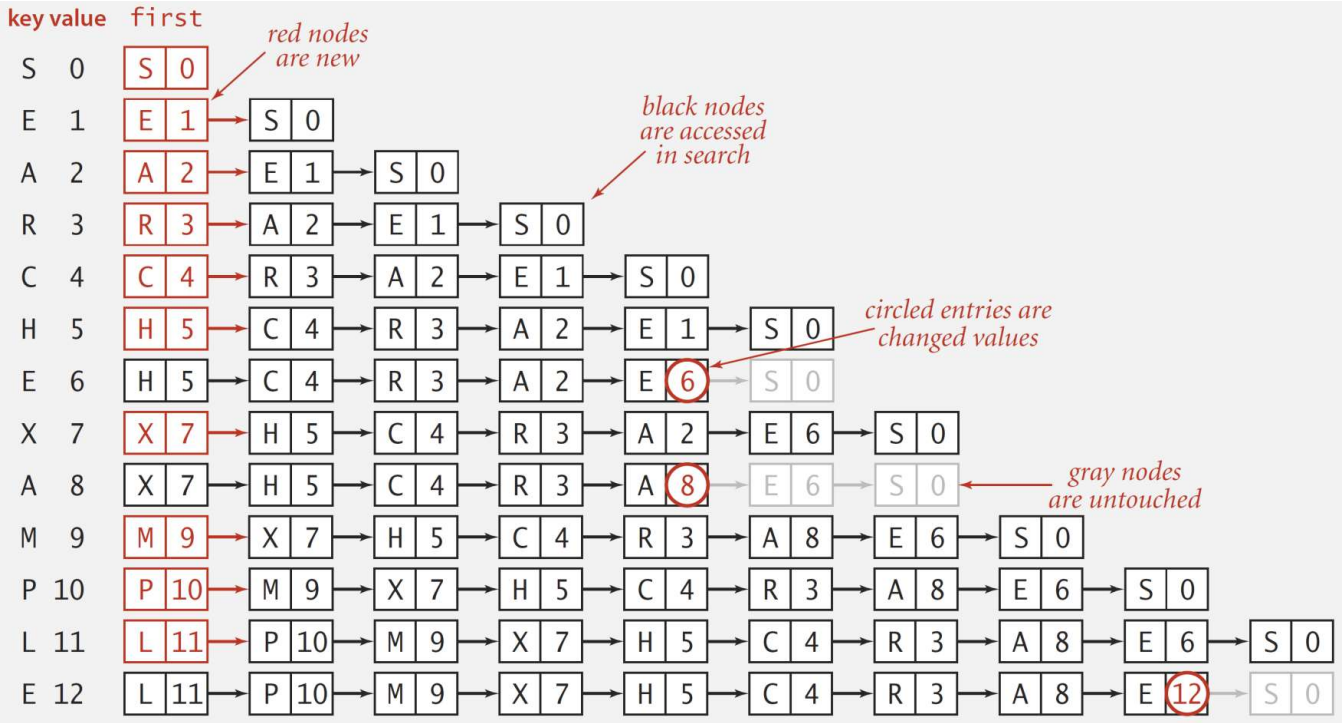
```
public class FrequencyCounter {
    public static void main(String[] args) {
        int minlen = Integer.parseInt(args[0]);
        …
        try {
            sc = new Scanner(file);
            long start = System.currentTimeMillis();      // 실행 시간 측정
            while (sc.hasNext()) {
                String word = sc.next();
                if (word.length() < minlen)    continue;
                if (!st.contains(word))   st.put(word, 1);
                else   st.put(word, st.get(word) + 1);
            }
            String maxKey = "";     int maxValue = 0;
            for (String word : st.keys())
                if (st.get(word) > maxValue) { maxValue = st.get(word);   maxKey = word; }
            long end = System.currentTimeMillis();
            System.out.println(maxKey + " " + maxValue);
            System.out.println("소요 시간 = " + (end-start) + "ms");
        } …
```

키: 단어
값: 단어의 빈도수
를 심볼 테이블에 저장

빈도수가 가장 높은 단어를
조사

1.3 연결 리스트를 이용한 순차 검색

- (키, 값)의 쌍으로 구성된 연결 리스트를 유지
  - 검색: 모든 키를 스캔하면서 입력 키와의 일치 여부 확인
  - 삽입: 모든 키를 스캔하면서 입력 키와의 일치 여부 확인, 일치하는 키가 없으면 리스트의 제일 앞에 (키, 값)의 쌍을 추가



구현(1)

```
class Node<K,V> {       // 하나의 노드를 표현
                        // (키, 값, 다음 노드를 가리키는 참조)의 쌍으로 구성
    K key;    V value;      Node<K,V> next;

    public Node(K key, V value, Node<K,V> next) {
        this.key = key;    this.value = value;   this.next = next;
}  }

public class SequentialSearchST<K,V> {
    private Node<K,V> first;       // 첫 번째 노드에 대한 참조를 유지. 초기값 = null
    int N;                         // 연결 리스트의 노드 수를 유지. 초기값 = 0

    public V get(K key)                {  다음 슬라이드 참조        }
    public void put(K key, V value)    {  다음 슬라이드 참조        }
    public void delete(K key)          {  다음 슬라이드 참조        }
    public Iterable<K> keys()          {  다음 슬라이드 참조        }
    public boolean contains(K key)     {  return get(key) != null;  }
    public boolean isEmpty()           {  return N == 0;            }
    public int size()                  {  return N;                 }
}
```

구현(2)

```
public V get(K key) {
    for (Node<K,V> x = first; x != null; x = x.next)   // 연결 리스트를 스캔
        if (key.equals(x.key))
            return x.value;          // search hit
    return null;                     // search miss
}

public void put(K key, V value) {
    for (Node<K,V> x = first; x != null; x = x.next)
        if (key.equals(x.key)) {     // 키가 있을 경우, 값만 변경
            x.value = value;
            return;
        }
    first = new Node<K,V>(key, value, first);    // 키가 없으면, 앞에 노드 추가
    N++;
}
```

구현(3)

```
public void delete(K key) {
    if (key.equals(first.key)) {     // 첫번째 노드를 삭제하는 경우
        first = first.next;   N--;
        return;
    }
    for (Node<K,V> x = first; x.next != null; x = x.next) {   // 삭제할 노드를 검색
        if (key.equals(x.next.key)) {
            x.next = x.next.next;    N--;
            return;
} } }

public Iterable<K> keys() {
    ArrayList<K> keyList = new ArrayList<K>(N);      // ArrayList는 Iterable을 구현
    for (Node<K,V> x = first; x != null; x = x.next)
        keyList.add(x.key);
    return keyList;
}
```

1.4 정렬된 배열을 이용한 이진 검색

- 키의 오름차순으로 (키,값)의 쌍을 배열에 저장

		keys[]											vals[]									
key	value	0	1	2	3	4	5	6	7	8	9	N	0	1	2	3	4	5	6	7	8	9
S	0	S										1	0									
E	1	E	S									2	1	0								
A	2	A	E	S								3	2	1	0							
R	3	A	E	R	S							4	2	1	3	0						
C	4	A	C	E	R	S						5	2	4	1	3	0					
H	5	A	C	E	H	R	S					6	2	4	1	5	3	0				
E	6	A	C	E	H	R	S					6	2	4	6	5	3	0				
X	7	A	C	E	H	R	S	X				7	2	4	6	5	3	0	7			
A	8	A	C	E	H	R	S	X				7	8	4	6	5	3	0	7			
M	9	A	C	E	H	M	R	S	X			8	8	4	6	5	9	3	0	7		
P	10	A	C	E	H	M	P	R	S	X		9	8	4	6	5	9	10	3	0	7	
L	11	A	C	E	H	L	M	P	R	S	X	10	8	4	6	5	11	9	10	3	0	7
E	12	A	C	E	H	L	M	P	R	S	X	10	8	4	12	5	11	9	10	3	0	7
		A	C	E	H	L	M	P	R	S	X		8	4	12	5	11	9	10	3	0	7

entries in red were inserted

entries in gray did not move

entries in black moved to the right

circled entries are changed values

구현(1)

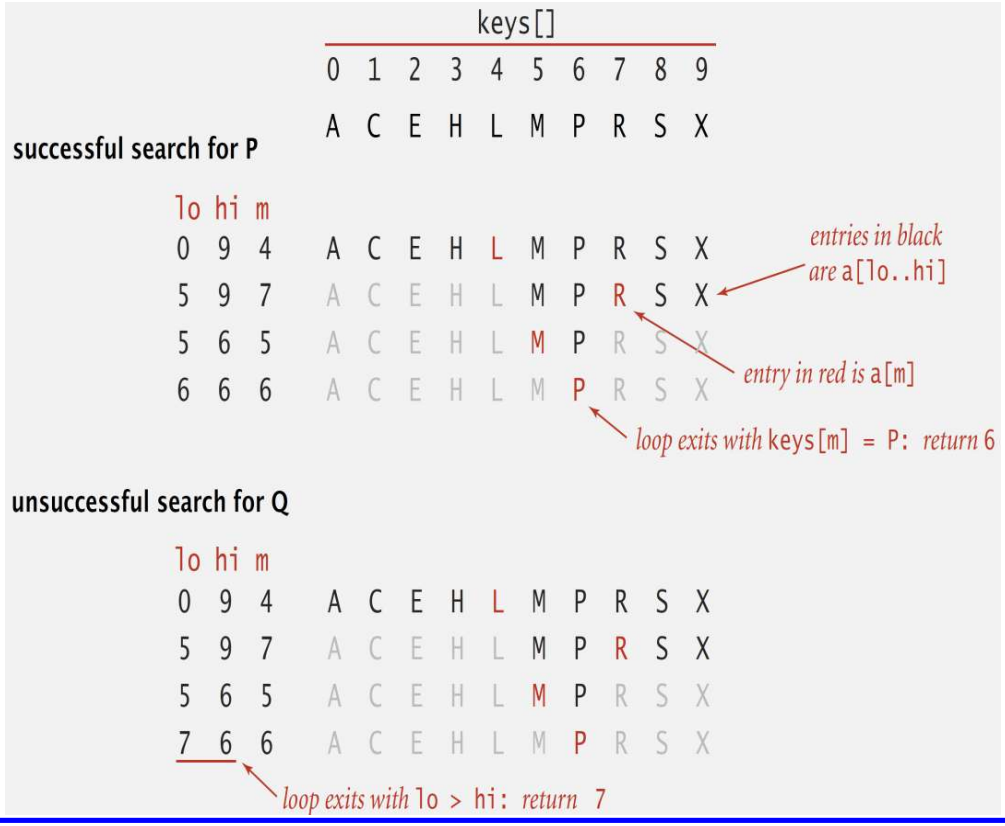
```
public class BinarySearchST<K extends Comparable<K>, V> {
    private static final int INIT_CAPACITY = 10;
    private K[] keys;      // 키의 배열
    private V[] vals;      // 값의 배열        ← 하나의 배열로 (키, 값) 쌍을 표현하는 방법?
    private int N;

    public BinarySearchST() {
        keys = (K[]) new Comparable[INIT_CAPACITY];
        vals = (V[]) new Object[INIT_CAPACITY];
    }

    public BinarySearchST(int capacity) {
        keys = (K[]) new Comparable[capacity];      // 배열 용량보다 많은 키가 입력?
        vals = (V[]) new Object[capacity];
    }

    public boolean contains(K key) {return get(key) != null; }
    public boolean isEmpty() {return N == 0; }
    public int size() { return N; }
```

구현(2)

```
private void resize(int capacity) {     // 배열 크기를 동적으로 변경
    K[] tempk = (K[]) new Comparable[capacity];
    V[] tempv = (V[]) new Object[capacity];
    for (int i = 0; i < N; i++) {
        tempk[i] = keys[i];
        tempv[i] = vals[i];
    }
    vals = tempv;
    keys = tempk;
}
```

구현(3)

```
private int search(K key) {     // 이진 검색
    int lo = 0;
    int hi = N – 1;
    while (lo <= hi) {
        int mid = (hi + lo) / 2;
        int cmp =
            key.compareTo(keys[mid]);

        if (cmp < 0)        hi = mid – 1;
        else if (cmp > 0)   lo = mid + 1;
        else                return mid;
    }
    // 키가 없을 경우, -1이 아니라 lo가 반환
    return lo;
}
```



구현(4)

```
public V get(K key) {
    if (isEmpty())  return null;
    int i = search(key);          // 이진 검색
    if (i < N && keys[i].compareTo(key) == 0)  return vals[i];
    else       return null;       // 키가 없으면 null을 반환
}

public void put(K key, V value) {
    int i = search(key);          // 일단 키를 찾고...
    if (i < N && keys[i].compareTo(key) == 0)      // 있으면, 값만 변경. 왜 비교를 다시?
    {       vals[i] = value; return; }
    if (N == keys.length)         // 없으면, 추가해야 하니 배열 크기 확장
        resize(2 * keys.length);

    for (int j = N; j > i; j--)
    {    keys[j] = keys[j-1]; vals[j] = vals[j-1]; }     // 추가될 곳의 공간 확보

    keys[i] = key; vals[i] = value;    N++;
}
```

구현(5)

```
public void delete(K key) {
    if (isEmpty())  return;
    int i = search(key);          // 이진 검색: 모든 키는 이진 검색으로 찾자!!!
    if (i == N || keys[i].compareTo(key) != 0)   return;      // 없으면, 그냥 반환

    for (int j = i; j < N-1; j++)
    {  keys[j] = keys[j+1];     vals[j] = vals[j+1];  }  // 뒤에 있는 키들을 한칸 앞으로.
    N--;
    keys[N] = null;    vals[N] = null;                    // 왜 null로 채울까?
    if (N > INIT_CAPACITY && N == keys.length/4)  // 배열이 너무 크면, 다시 줄이자.
        resize(keys.length/2);
}

public Iterable<K> keys() {       // 연결 리스트의 경우와 거의 동일
    ArrayList<K> keyList = new ArrayList<K>(N);
    for (int i = 0; i < N; i++)
        keyList.add(keys[i]);
     return keyList;
}
```

성능 비교

알고리즘	최악의 성능 (N개의 삽입 후)		평균 성능 (N개의 삽입 후)		ordered operation 지원?
	검색	삽입	검색	삽입	
순차검색 (연결 리스트)	N	N	N/2	N	no
이진 검색 (정렬된 배열)	logN	2N	logN	N	yes

FrequencyCounter의 실행 결과

	순차 검색	이진 검색
tinyTale.txt	6 ms	2 ms
tale.txt	1,054 ms	149 ms
leipzig1M.txt	INF	203,874ms

* 8자 미만의 단어들은 무시

이 시간을 줄이는 방법?

1.5 Ordered Operations

Ordered symbol table API의 예

	keys	values
min() →	09:00:00	Chicago
	09:00:03	Phoenix
	09:00:13	Houston
get(09:00:13) →	09:00:59	Chicago
	09:01:10	Houston
floor(09:05:00) →	09:03:13	Chicago
	09:10:11	Seattle
select(7) →	09:10:25	Seattle
	09:14:25	Phoenix
	09:19:32	Chicago
	09:19:46	Chicago
keys(09:15:00, 09:25:00) →	09:21:05	Chicago
	09:22:43	Seattle
	09:22:54	Seattle
	09:25:52	Chicago
ceiling(09:30:00) →	09:35:21	Chicago
	09:36:14	Seattle
max() →	09:37:44	Phoenix

size(09:15:00, 09:25:00) *is* 5

rank(09:10:25) *is* 7

Ordered Symbol Table API

public class ST<Key extends Comparable<Key>, Value>

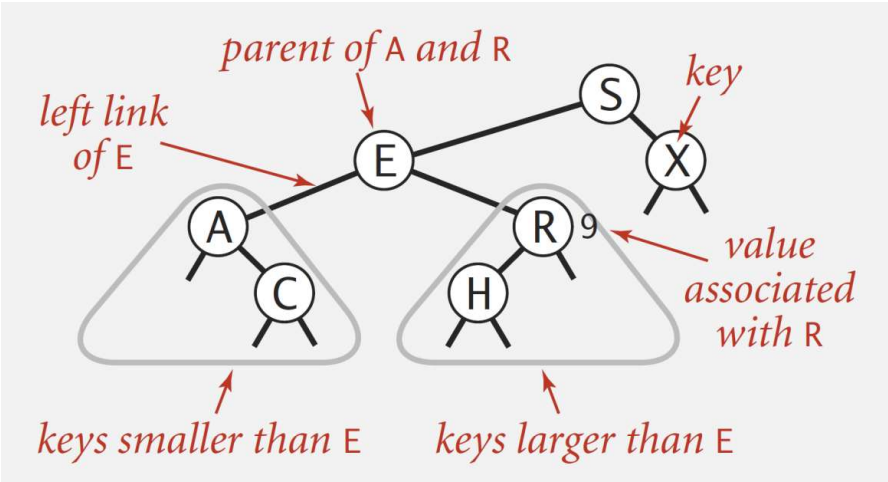
…	
Key min()	가장 작은 키
Key max()	가장 큰 키
Key floor(Key k)	"키 <= k" 중에서 제일 큰 키
Key ceiling(Key k)	"k <= 키" 중에서 제일 작은 키
int rank(Key k)	k보다 작은 키의 수
Key select(int r)	랭크가 r인 키
void deleteMin()	가장 작은 키를 삭제
void deleteMax()	가장 큰 키를 삭제
int size(Key lo, Key hi)	lo와 hi 사이에 있는 키의 수
Iterable<Key> keys()	정렬된 순서로 모든 키를 반환
Iterable<Key> keys(Key lo, Key hi)	정렬된 순서로 lo~hi 사이의 키를 반환

순차 검색과 이진 검색의 성능

	순차 검색	이진 검색
get	N	logN
put/delete	N	N
min/max	N	1
floor/ceiling	N	logN
rank	N	logN
select	N	1
ordered iteration	N logN	N

2. Binary Search Tree (BST)

- BST의 정의
  - 각 노드는 (키, 값)의 쌍을 저장하며, 서로 다른 노드의 키는 다르다.
  - 부모의 키는 왼쪽 서브트리에 저장된 노드의 키보다 크다.
  - 부모의 키는 오른쪽 서브트리에 저장된 노드의 키보다 작다.



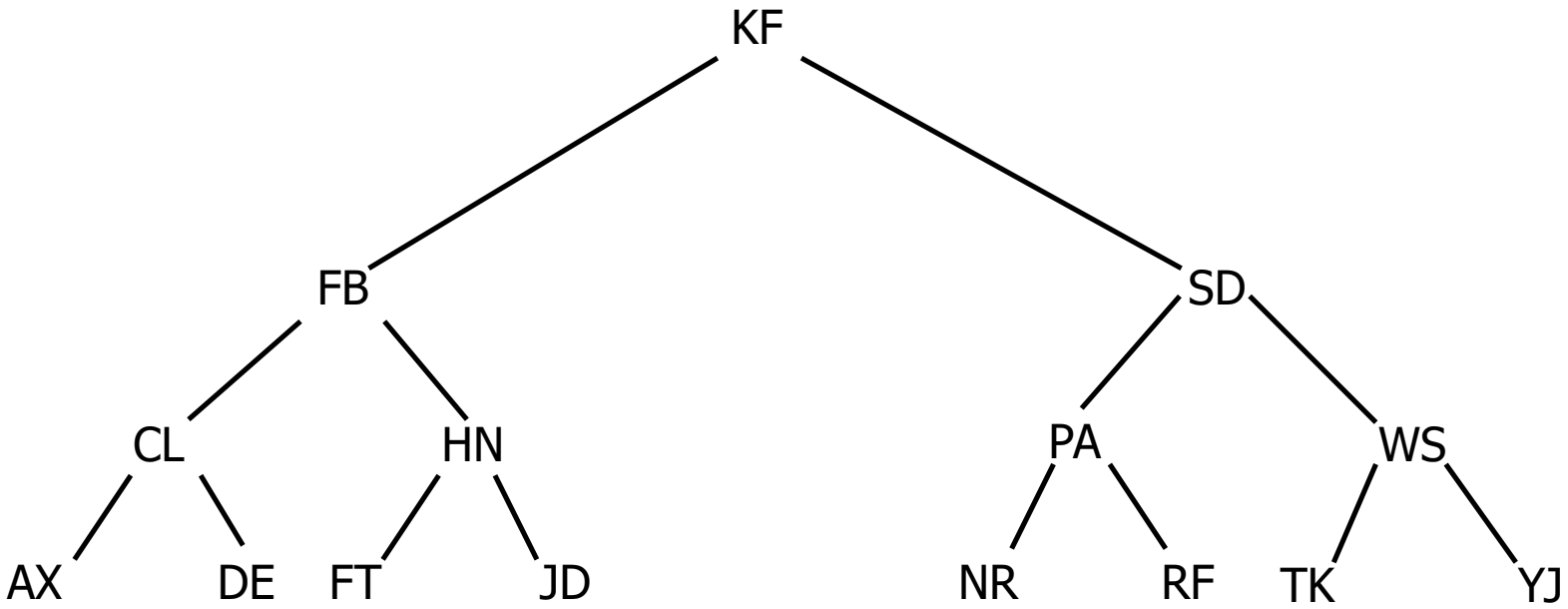
BST의 동작 과정

- 동작 과정
  - 검색: 루트부터 비교.
    - 작으면 왼쪽, 크면 오른쪽. 같으면 발견.
  - 삽입: 검색 후, 있으면 수정. 없으면 추가.

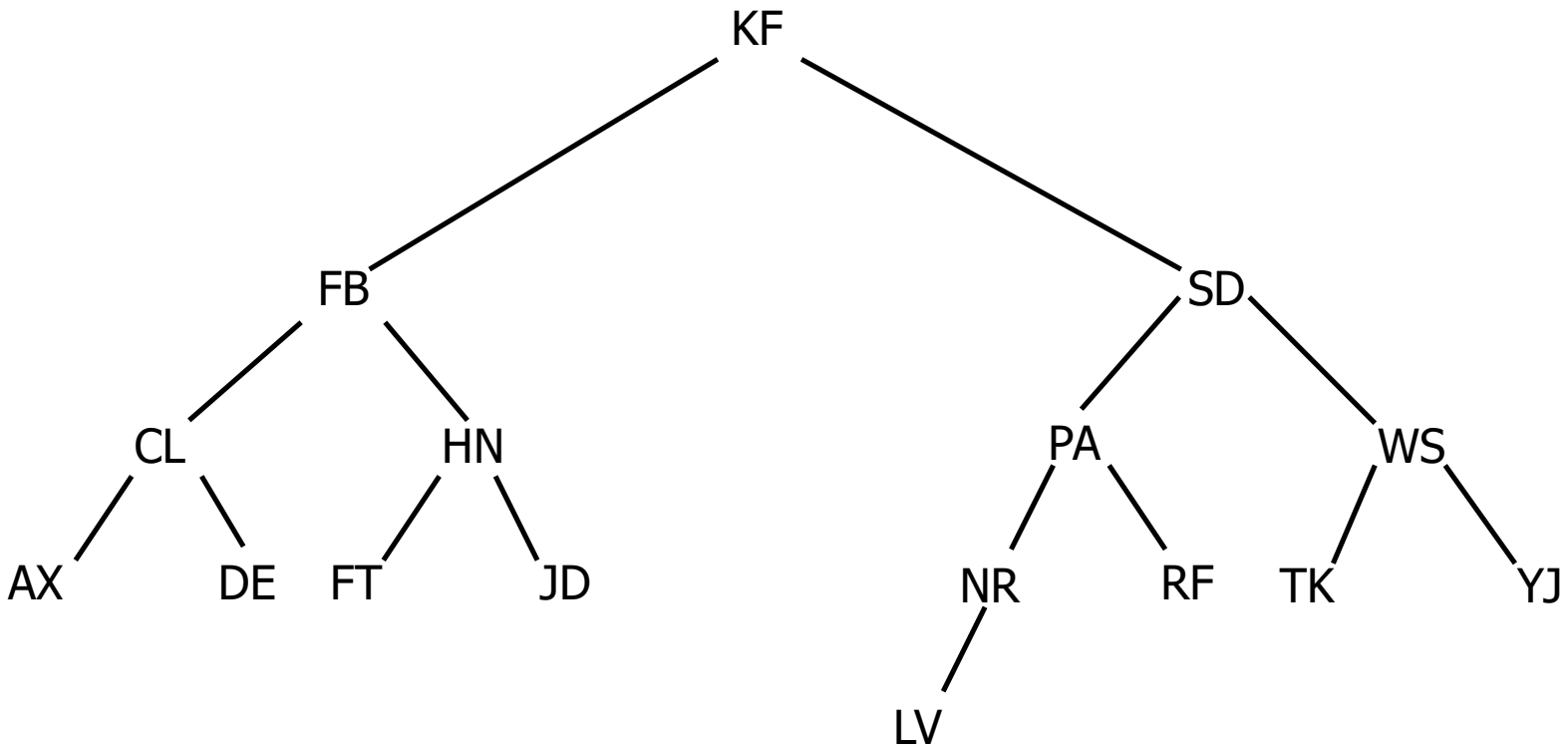
- 장점
  - 심볼 테이블을 정렬된 순서로 유지할 필요 없다.
  - 새로 삽입되는 노드는 항상 리프 노드에 위치

- 문제점
  - Balanced tree를 지원하지 못함
  - Too many seeks
    - 24개의 key 로 구성된 tree: 5번의 seek

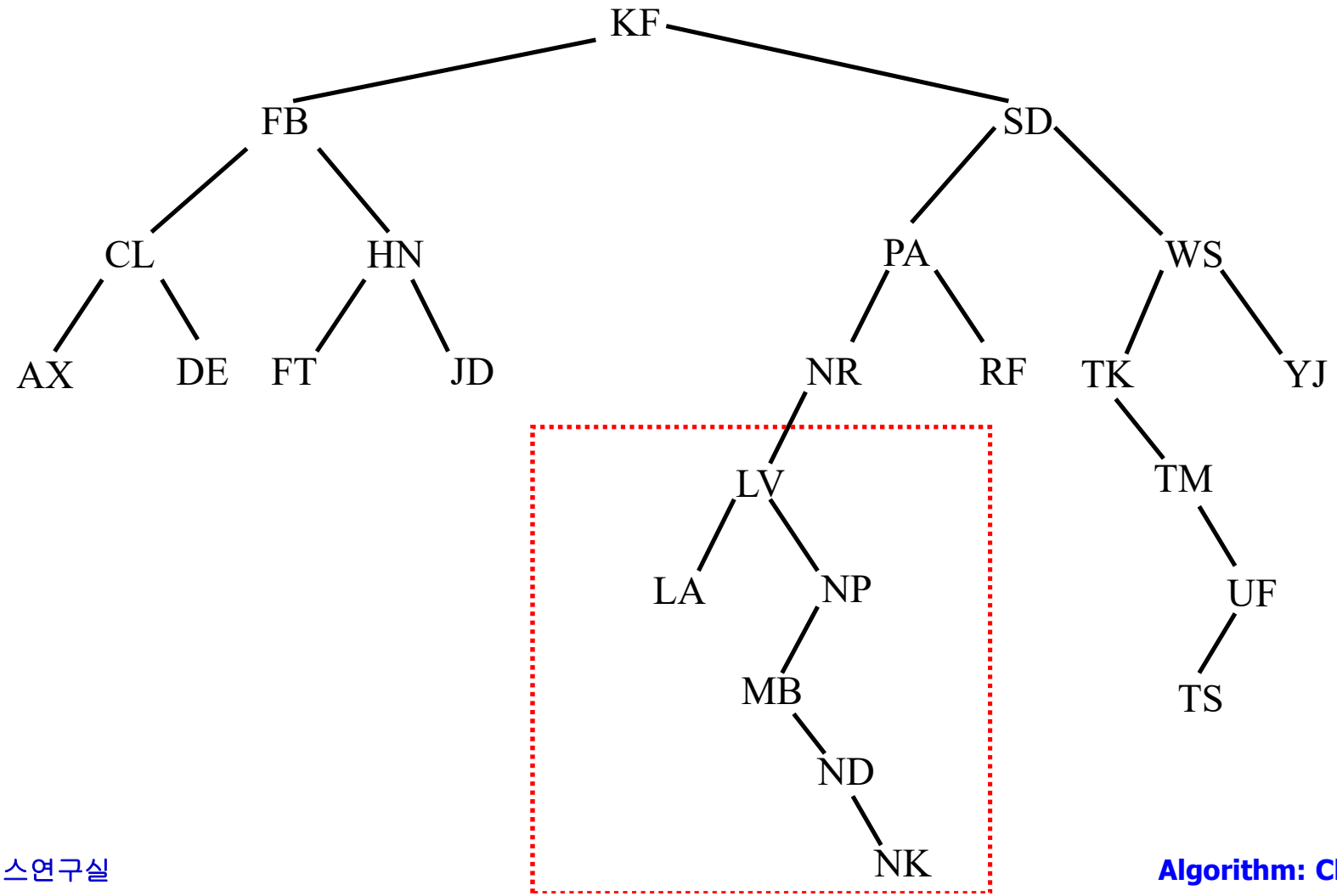
BST의 예



LV가 추가된 BST



여러 개의 키가 추가된 BST



Java를 이용한 BST의 구현

- Node와 트리(BST)를 구분하여 구현
- Node: key와 value, 자식 노드에 대한 reference
- BST: root 노드(Node)에 대한 reference를 포함
  - 트리에 대한 연산들은 BST에서 구현

```
class Node<K,V> {
    K key;
    V value;
    Node<K,V> left, right;

    public Node(K key, V val) {
        this.key = key;
        this.value = val;
    }
}
```

기본적인 구조

+

```
class Node<K,V> {
    int N;     // 자손 노드 + 1 (ordered 연산)
    int aux:  // AVL 트리나 RB 트리에 사용
    Node<K,V> parent; // AVL or RB
    public Node(K key, V val) {
        this.key = key; this.value = val;
        this.N = 1;
    }
    public int getAux(){ return aux; }
    public void setAux(int value) { aux = value; }
}
```

Ordered operation과 **Balanced Tree**를 위한 추가적인 구조

BST 클래스

```
public class BST<K extends Comparable<K>, V> {
    protected Node<K,V> root;
    public int size() { return (root != null) ? root.N : 0; }
    public V get(K key) { … }
    public void put(K key, V val) { … }
    public Iterable<K> keys() { … }
    public void delete(K key) { … }
    public K min() { … }
    public K max() { … }
    public K floor(K key) { … }
    public K ceiling(K key) { … }
    public K select(int rank) { … }
    public int rank(K key) { … }
    public void deleteMin() { delete(min()); }
    public void deleteMax() { delete(max()); }
    public int size(K lo, K hi) { … }
    public Iterable<K> keys(K lo, K hi) { … }
}
```

Elementary Operations

Ordered Operations

BST 구현 – treeSearch()

- 키를 입력받아, 그 키를 갖는 노드 또는 순회의 마지막 노드를 반환

```
protected Node<K,V> treeSearch(K key) {
    Node<K,V> x = root;        // BST에 대한 모든 연산은 루트부터 시작
    while (true) {
        int cmp = key.compareTo(x.key);
        if (cmp == 0)          return x;        // 찾았으면, 순회 종료
        else if (cmp < 0) {                     // x.key보다 작을 경우, 왼쪽으로
            if (x.left == null)  return x;      // 없으면, 순회 종료
            else               x = x.left;
        }
        else {                                  // x.key보다 클 경우, 오른쪽으로
            if (x.right == null) return x;      // 없으면, 순회 종료
            else               x = x.right;
        }
    }
}
```

비교 횟수 = 노드의 레벨 (단, 루트의 레벨 = 1)

BST 구현 – get()

- Get: 주어진 키에 해당하는 값을 반환. 키가 없으면 null

```
public V get(K key) {
   if (root == null) return null;
   Node<K,V> x = treeSearch(key);
   if (key.equals(x.key))    // 검색 키를 가진 노드가 반환된 경우
      return x.value;
   else                      // 검색 키를 가진 노드가 없는 경우
      return null;
}
```

BST 구현 – put()

- Put: 검색 후, 키가 있으면 reset. 없으면 추가.

```
public void put(K key, V val) {
   if (root == null)    {  root = new Node<K,V>(key, val);  return;  }
   Node<K,V> x = treeSearch(key);
   int cmp = key.compareTo(x.key);
   if (cmp == 0)    x.value = val;    // 키가 존재하므로, 값을 reset
   else {                             // 없는 키: 새로운 노드를 생성하여 x의 자식으로 추가
      Node<K,V> newNode = new Node<K,V>(key, val);
      if (cmp < 0)   x.left = newNode;
      else           x.right = newNode;
      newNode.parent = x;
      rebalanceInsert(newNode);     // Insert의 후속 조치: 다음 slide
   }
}
```

비용: 비교 횟수 = 노드의 레벨

BST 구현 – rebalanceInsert()

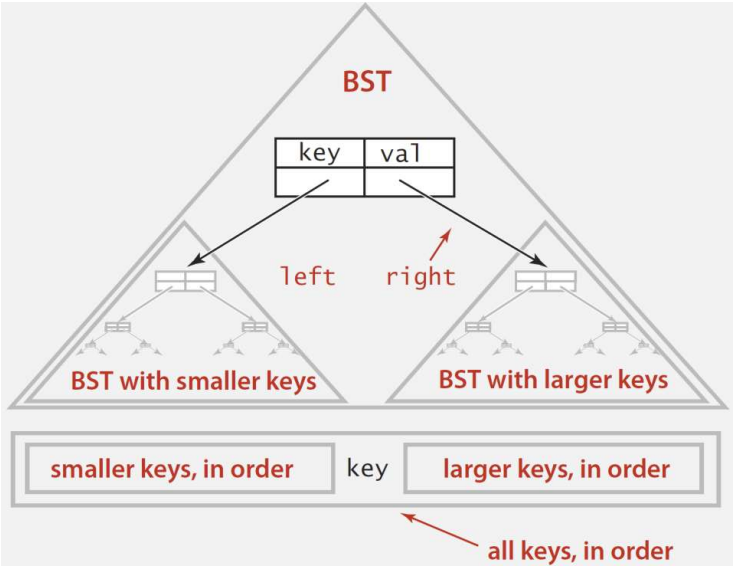
- BST의 subclass인 AVLTree나 RBTree에서 overriding
  - BST에서는 신규 노드의 모든 조상 노드들의 size 증가

```
protected void rebalanceInsert(Node<K,V> x) {
   resetSize(x.parent, 1);  // root까지 조상 노드들의 size를 1 증가
}

protected void rebalanceDelete(Node<K,V> p, Node<K,V> deleted) {
   resetSize(p, -1);          // root까지 조상 노드들의 size를 1 감소
}

private void resetSize(Node<K,V> x, int value) {
   for ( ; x != null; x = x.parent)
      x.N += value;
}
```

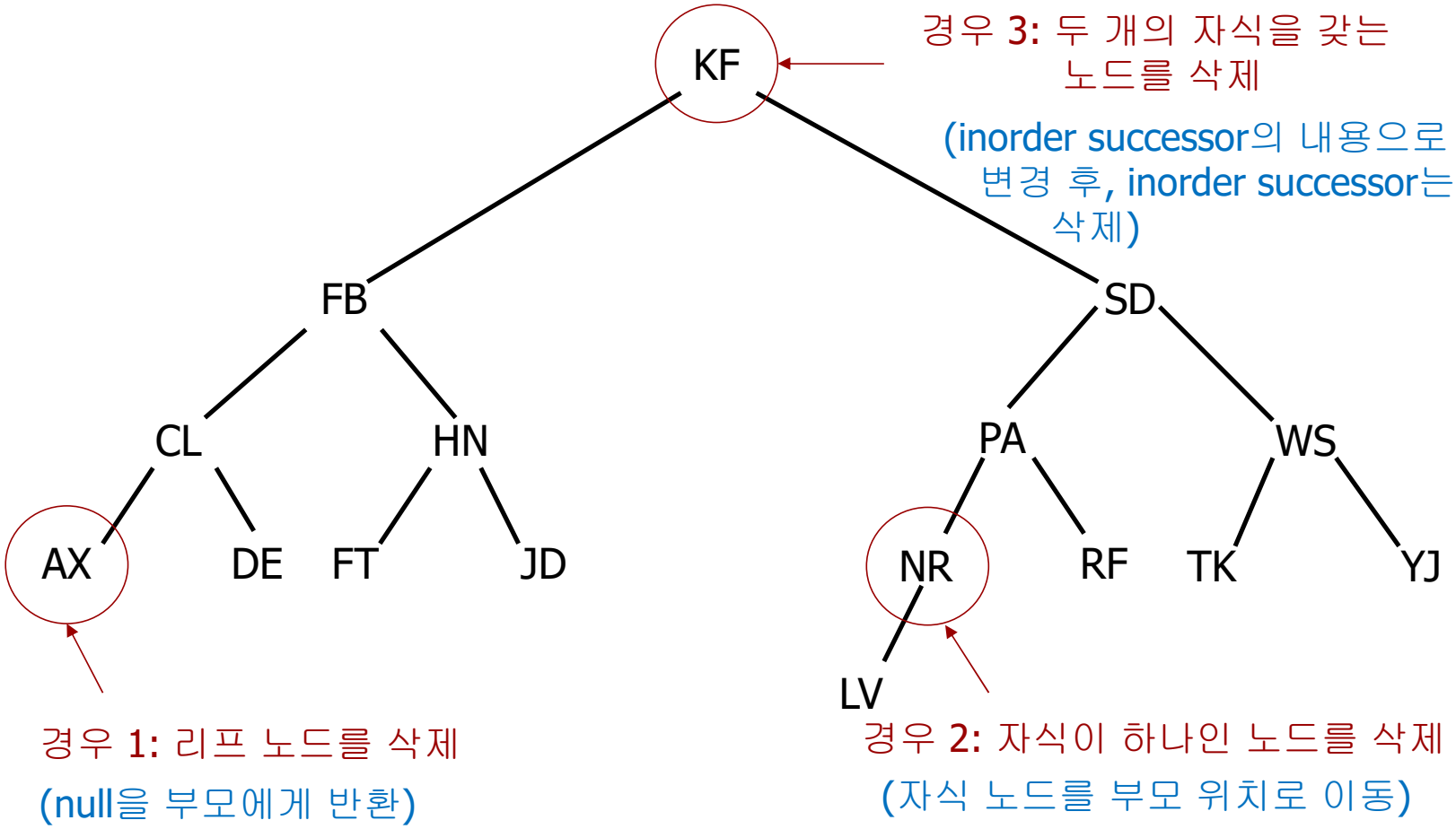
BST 구현 – keys()

- 키의 정렬된 리스트를 반환 → Inorder 순회 이용

```
public Iterable<K> keys() {
    if (root == null)    return null;
    ArrayList<K> keyList = new ArrayList<K>(size());
    inorder(root, keyList);
    return keyList;
}

private void inorder(Node<K,V> x, ArrayList<K> keyList) {
    if (x != null) {
        inorder(x.left, keyList);
        keyList.add(x.key);
        inorder(x.right, keyList);
    }
}
```



BST 구현 – delete()　　단계1: 개념



BST 구현 – delete() 단계2: 구현

```
public void delete(K key) {
    if (root == null)  return;
    Node<K,V> x, y, p;
    x = treeSearch(key);

    // key가 없는 경우.
    if (!key.equals(x.key))
        return;

    // 루트이거나 자식이 두 개인 노드
    if (x == root || isTwoNode(x)) {
        if (isLeaf(x))    // 루트가 리프
            { root = null; return; }
        else if (!isTwoNode(x)) {   // 루트
            root = (x.right == null) ?
                x.left : x.right;     // 자식을 루트로
            root.parent = null;
            return;
        }
        else {    // 자식이 둘인 노드(루트 포함)
            y = min(x.right);      // inorder successor
            x.key = y.key;         // y를 x에 복사
            x.value = y.value;     // y를 x에 복사
            p = y.parent;
            // y의 자식을 p의 자식으로(y 삭제)
            relink(p, y.right, y == p.left);  // next page
            // y의 조상 노드들의 size를 감소
            rebalanceDelete(p, y);
        }
    }
    else {    // 자식 ≤ 1 이고, 루트 아님
        p = x.parent;
        if (x.right == null)
            relink(p, x.left, x == p.left);
        else if (x.left == null)
            relink(p, x.right, x == p.left);
        rebalanceDelete(p, x);
    }
}
```

BST 구현 – inline과 protected 함수들

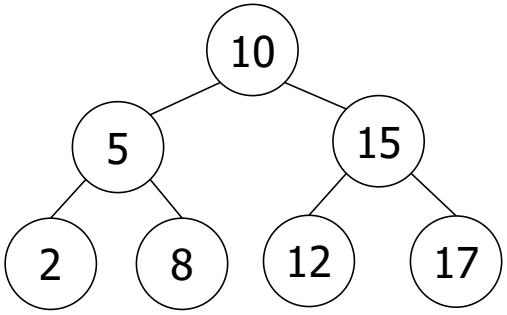
```
public boolean contains(K key) {return get(key) != null; }
public boolean isEmpty() {return root == null; }

protected boolean isLeaf(Node<K,V> x)
{    return x.left == null && x.right == null; }

protected boolean isTwoNode(Node<K,V> x)
{    return x.left != null && x.right != null; }

protected void relink(Node<K,V> parent, Node<K,V> child, boolean makeLeft) {
     if (child != null)   child.parent = parent;    // child를 parent의 자식으로
     if (makeLeft)  parent.left = child;            // 왼쪽 자식, 또는
     else           parent.right = child;           // 오른쪽 자식
}

protected Node<K,V> min(Node<K,V> x) { while (x.left != null)    x = x.left;      return x;  }
```

BST 구현 – min()과 max()

```
public K min() {              // 제일 작은 키를 반환
   if (root == null)  return null;
   Node<K,V> x = root;
   while (x.left != null)     // 제일 왼쪽에 있는 노드
      x = x.left;
   return x.key;
}

public K max() {              // 제일 큰 키를 반환
   if (root == null)  return null;
   Node<K,V> x = root;
   while (x.right != null)    // 제일 오른쪽에 있는 노드
      x = x.right;
   return x.key;
}
```

BST 구현 – floor()

```
public K floor(K key) {   // key보다 작거나 같은 키들 중에서 제일 큰 키
    if (root == null || key == null)  return null;
    Node<K,V> x = floor(root, key);
    if (x == null)     return null;
    else               return x.key;
}

private Node<K,V> floor(Node<K,V> x, K key) {
    if (x == null)     return null;
    int cmp = key.compareTo(x.key);
    if (cmp == 0)      return x;                  // key와 동일한 키를 가진 노드
    if (cmp <  0)      return floor(x.left, key); // key보다 크다면 계속 왼쪽으로
    Node<K,V> t = floor(x.right, key);            // key가 클 경우, 오른쪽으로
    if (t != null)     return t;                  // 오른쪽에 작은 키가 있을 경우
    else               return x;                  // 오른쪽에 작은 키가 없을 경우
}
```

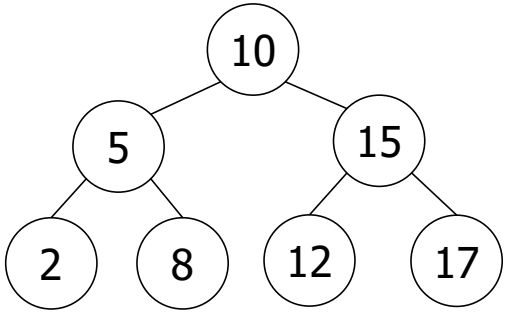


floor(10)
floor(9)
floor(13)

BST 구현 – rank()

```
public int rank(K key) {      // key보다 작은 키의 수
    if (root == null || key == null)  return 0;
    Node<K,V> x = root;
    int num = 0;
    while (x != null) {         // 루트부터 비교하면서 key보다 작은 키의 수를 합산
        int cmp = key.compareTo(x.key);
        if (cmp < 0)  x = x.left;
        else if (cmp > 0) {             // key보다 작은 키를 갖는 노드를 발견
            num += 1 + size(x.left);    // 왼쪽 subtree의 노드 수를 합산
            x = x.right;                // 오른쪽 subtree도 계속 검사
        }
        else {                          // key값을 갖는 노드: 왼쪽 subtree만 합산
            num += size(x.left);  break;
        }
    }
    return num;
}
```

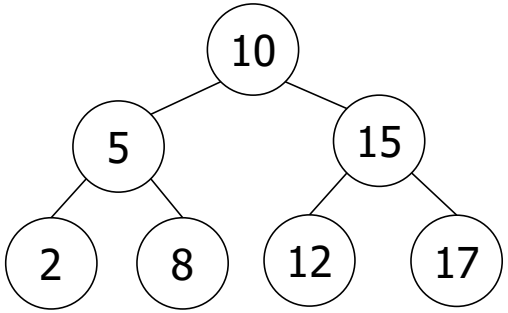
```
private int size(Node<K,V> x) { return (x != null) ? x.N : 0; }
```



rank(10)
rank(7)
rank(13)

BST 구현 – select()

```
public K select(int rank) {      // rank 등수에 해당하는 키를 반환
    if (root == null || rank < 0 || rank >= size())
        return null;
    Node<K,V> x = root;
    while (true) {
        int t = size(x.left);
        if (rank < t)            // 왼쪽 subtree의 노드 수가 rank보다 크면
            x = x.left;          // rank 등수의 키는 당연히 왼쪽 subtree에 있겠지
        else if (rank > t) {     // rank가 왼쪽 subtree의 노드 수 밖이면
            rank = rank - t - 1; // 왼쪽 subtree와 부모 노드는 결과에서 제외하고
            x = x.right;         // 오른쪽 subtree를 조사하자
        }
        else                     // 왼쪽 subtree의 수와 rank가 일치하면
            return x.key;        // 부모 노드의 키를 반환 (rank의 시작은 0)
    }
}
```



select(0)
select(2)
select(5)

BST의 성능

	순차 검색	이진 검색	**BST**
get	N	logN	h
put/delete	N	N	h
min/max	N	1	h
floor/ceiling	N	logN	h
rank	N	logN	h
select	N	1	h
ordered iteration	N logN	N	N

최악의 경우: h = N
무작위 입력의 경우: h = 1.39 logN

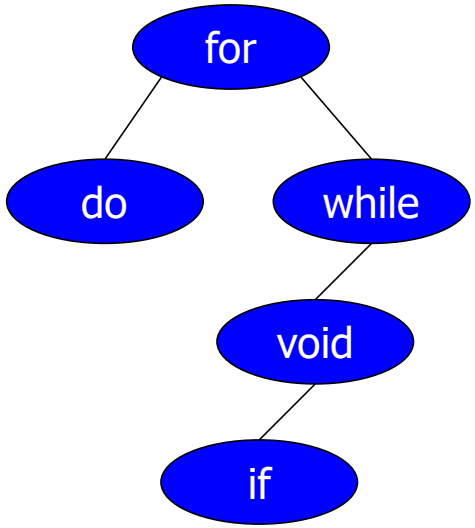
FrequencyCounter의 실행 결과

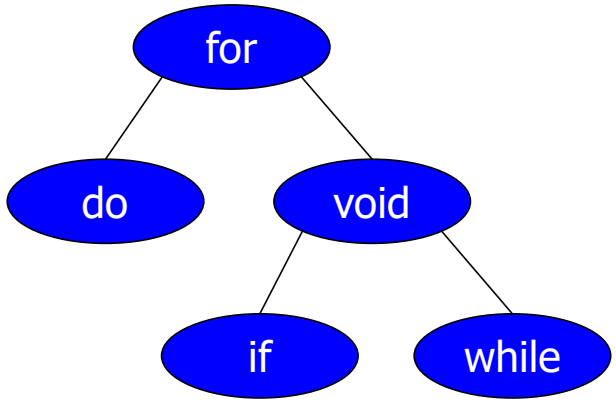
	tinyTale.txt	tale.txt	leipzig1M.txt
순차 검색	6ms	1,054ms	INF
이진 검색	2ms	149ms	203,874ms
BST	0ms	124ms	10,951ms

* 8자 미만의 단어들은 무시

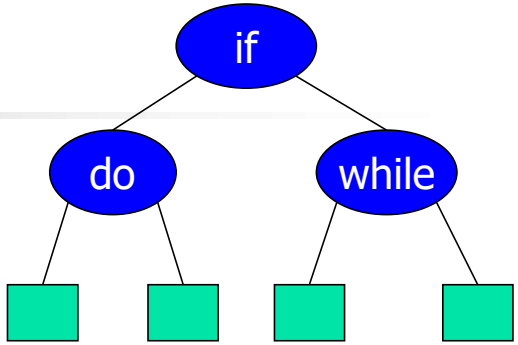
3. Optimal Binary Search Trees

- 정의
  - BST에서 키 값들의 average search length를 최소로!
  - 각 키에 대한 access probability는 다를 수 있음.



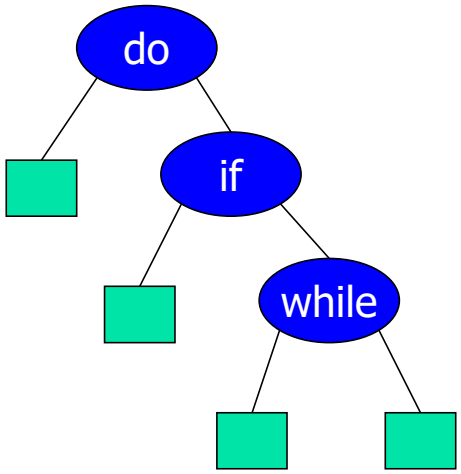


Cost of Binary Search Tree

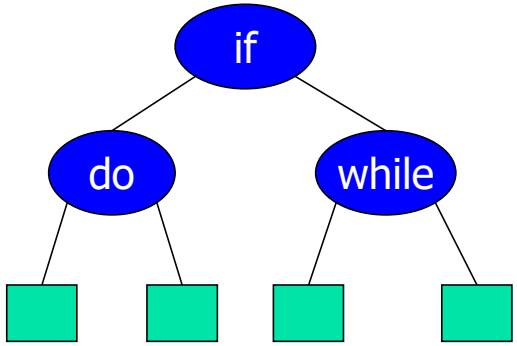


- Key의 access probability가 동일할 경우
  - 최소 비용 트리 = Complete binary tree

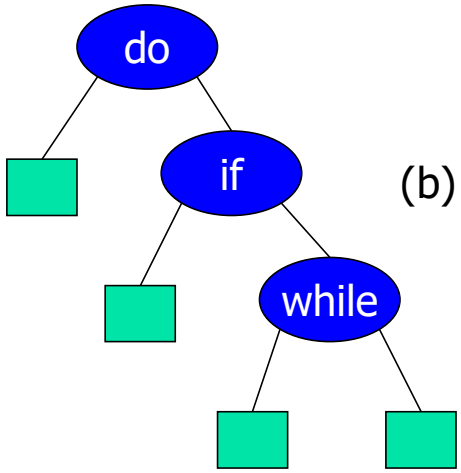
- Key의 access probability가 상이할 경우
  - Key 값: $a_1, a_2, \ldots, a_n$ $(a_1 < a_2 < \ldots < a_n)$
  - p_i: probability of searching a_i
  - E_i: all identifiers x such that $a_i < x < a_{i+1}$
  - q_i: 찾고자 하는 key값이 E_i에 포함될 확률
  - Cost: $\Sigma_{i=1} p_i \bullet \text{level}(a_i) + \Sigma_{i=0} q_i \bullet (\text{level(failure node } i) - 1)$
  - $\Sigma_{i=1} p_i + \Sigma_{i=0} q_i = 1$



Example



(a)



(b)

- $p_i = q_j = 1/7$ for all i and j
  - cost(tree a) = 13/7, cost(tree b) = 15/7

- $p_1 = .5, p_2 = .1, p_3 = .05, q_0 = .15, q_1 = .1, q_2 = .05, q_3 = .05$
  - cost(tree a) = 2*0.5+1*0.1+2*0.05+2*(0.15+0.1+0.05+0.05) = 1.9
  - cost(tree b) = 1*0.5+2*0.1+3*0.05+0.15+2*0.1+3*(0.05+0.05) = 1.5

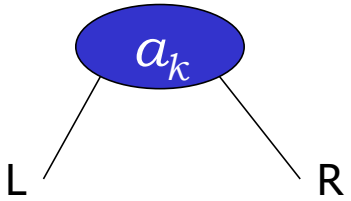
Finding Optimal Binary Search Tree

- 방법 1: 가능한 모든 BST에 대해 cost 계산
  - Complexity = $O(n\,N(n))$
  - $N(n)$: 노드 수가 n인 BST의 종류 = $O(4^n/n^{3/2})$

- 방법 2: Dynamic Programming
  - Let n identifiers: $a_1 < a_2 < \ldots < a_n$
  - T_{ij}: optimal BST for $a_{i+1}, \ldots, a_j$, $i < j$ $(a_k < a_{k+1})$
    - r_{ij}: root of T_{ij}
    - w_{ij}: weight of $T_{ij} = q_i + \Sigma(q_k + p_k)$, $i+1 \le k \le j$
    - By definition, r_{ii} = 0 and w_{ii} = q_i, $0 \le i \le n$
  - Goal: Find T_{0n}!

T_{0n}을 구하는 방법

- T_{ij}: optimal BST for $a_{i+1}, \ldots, a_j$ and $r_{ij} = a_k$ $(i < k \leq j)$



- Cost $c_{ij} = p_k$ + cost(L) + cost(R) + weight(L) + weight(R)
  - weight(L) = weight($T_{i,\,k\text{-}1}$) = $w_{i,\,k\text{-}1}$
  - weight(R) = weight(T_{kj}) = w_{kj}
- $c_{ij} = p_k + c_{i,\,k\text{-}1} + c_{kj} + w_{i,\,k\text{-}1} + w_{kj} = w_{ij} + c_{i,\,k\text{-}1} + c_{kj}$
- $w_{ij} + c_{i,\,k\text{-}1} + c_{kj} = \min\{w_{ij} + c_{i,\,l\text{-}1} + c_{lj}\},\ i < l \leq j$
- $\boldsymbol{c_{i,\,k\text{-}1} + c_{kj} = \min\{c_{i,\,l\text{-}1} + c_{lj}\},\ i < l \leq j}$

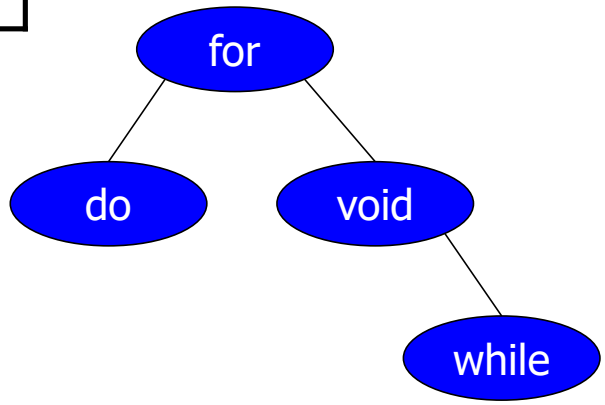
Function to find an optimal binary search tree

```
obst(int p[], int q[], int cost[][M], int root[][M], int weight[][M], int n)
{
    int    i, j, k, m, min, minpos;
    for (i = 0; i < n, i++) {  // 초기 조건 설정
            weight[i][i] = q[i];        root[i][i] = cost[i][i] = 0;
            cost[i][i+1] = weight[i][i+1] = q[i] + q[i+1] + p[i+1];
            root[i][i+1] = i + 1;
    }
    weight[n][n] = q[n]; root[n][n] = cost[n][n] = 0;
    for (m = 2; m <= n; m++)
        for (i = 0; i <= n – m; i++) {
            j = i + m;
            weight[i][j] = weight[i][j-1] + p[j] + q[j];
            k = knuth_min(cost, root, i, j);
            cost[i][j] = weight[i][j] + cost[i][k-1] + cost[k][j];
            root[i][j] = k;
        }
}
```

Example

- n = 4 and (a_1, a_2, a_3, a_4) = (**do**, **for**, **void**, **while**)
- (p_1, p_2, p_3, p_4) = (3, 3, 1, 1)
- $(q_0, q_1, q_2, q_3, q_4)$ = (2, 3, 1, 1, 1)

W_{00} = 2 C_{00} = 0 R_{00} = 0	W_{11} = 3 C_{11} = 0 R_{11} = 0	W_{22} = 1 C_{22} = 0 R_{22} = 0	W_{33} = 1 C_{33} = 0 R_{33} = 0	W_{44} = 1 C_{44} = 0 R_{44} = 0
W_{01} = 8 C_{01} = 8 R_{01} = 1	W_{12} = 7 C_{12} = 7 R_{12} = 2	W_{23} = 3 C_{23} = 3 R_{23} = 3	W_{34} = 3 C_{34} = 3 R_{34} = 4	
W_{02} = 12 C_{02} = 19 R_{02} = 1	W_{13} = 9 C_{13} = 12 R_{13} = 2	W_{24} = 5 C_{24} = 8 R_{24} = 3		
W_{03} = 14 C_{03} = 25 R_{03} = 2	W_{14} = 11 C_{14} = 19 R_{14} = 2			
W_{04} = 16 C_{04} = 32 R_{04} = 2				



- T02를 계산 (a1과 a2 두 개의 키로 구성)
  - w02 = w01 + p2 + q2 = 8 + 3 + 1 = 12
  - c02를 계산하기 위하여 a1→a2인 트리와 a1←a2인 트리를 비교
    - c02 = w02 + min(a1→a2, a1←a2)
      - w02 + c00 + c12 = 12 + 0 + 7 = 19
      - w02 + c01 + c22 = 12 + 8 = 20
    - r02 = 1
- T13, T24를 이런 방식으로 계산
- T03를 계산 (a1, a2, a3 세 개의 키로 구성)
  - w03 = w02 + p3 + q3 = 12 + 1 + 1 = 14
  - c03를 계산하기 위하여 a1→a2a3, a1←a2→a3, a1a2←a3인 트리를 비교
    - c03 = w03 + min(a1→a2a3, a1←a2→a3, a1a2←a3)
      - w03 + c00 + c13 = 14 + 0 + 12 = 26
      - w03 + c01 + c23 = 14 + 8 + 3 = 25
      - w03 + c02 + c33 = 14 + 19 + 0 = 33
    - r03 = 2

연습 문제

- 4개의 키 값과 그 키 값을 검색할 확률이 아래 표와 같을 때 질문에 답하시오.

순서	키 값	검색 확률(p_i)
1	Twitter	0.05
2	Google	0.5
3	Apple	0.2
4	Facebook	0.25

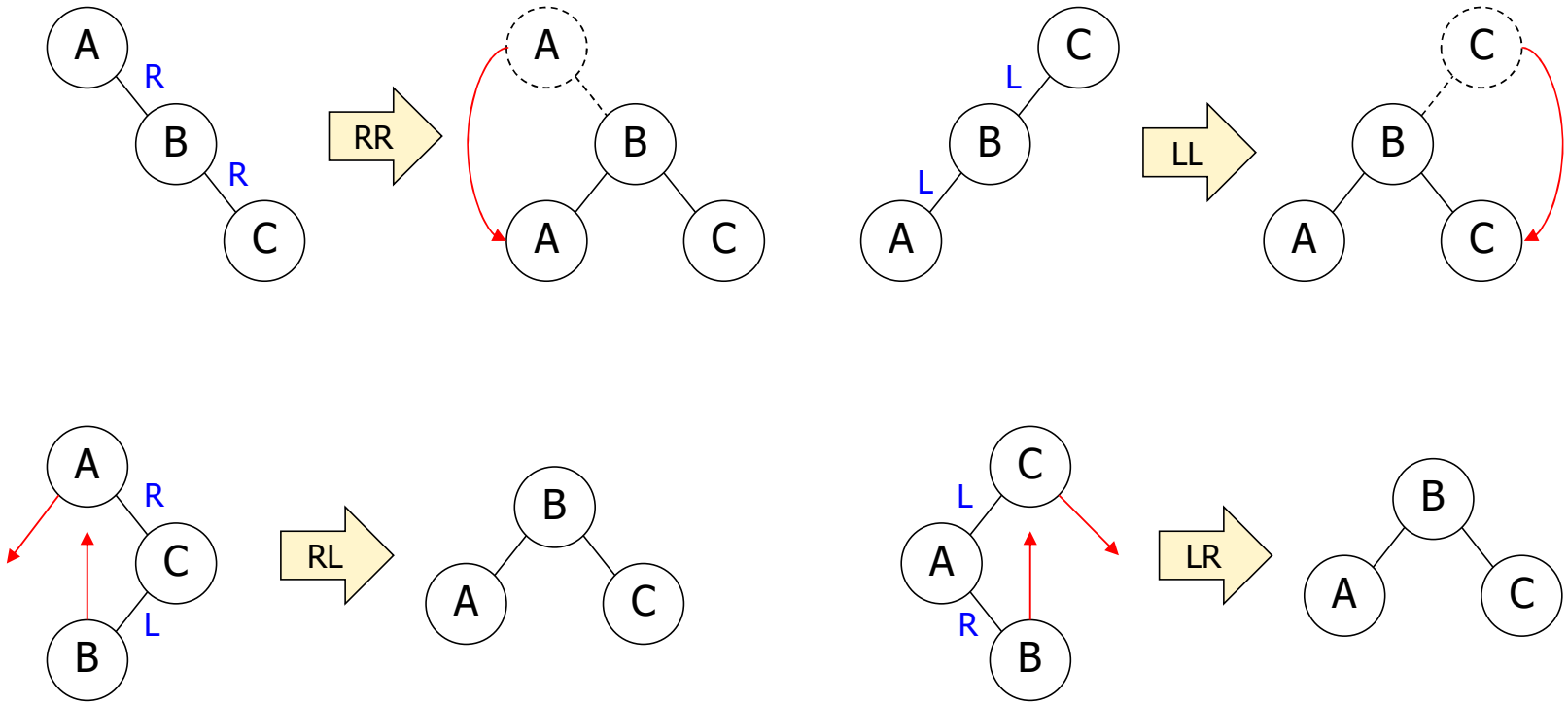
  - 표의 키 값들이 순서대로 입력될 경우의 이진검색 트리(binary search tree)를 작성하고, 이 트리의 총 검색 비용을 계산하시오.
  - 표의 키 값에 대해 검색 비용이 최소가 되는 최적 이진검색 트리(optimal binary search tree)를 작성하고, 이 트리의 총 검색 비용을 계산하시오.

4. AVL Trees

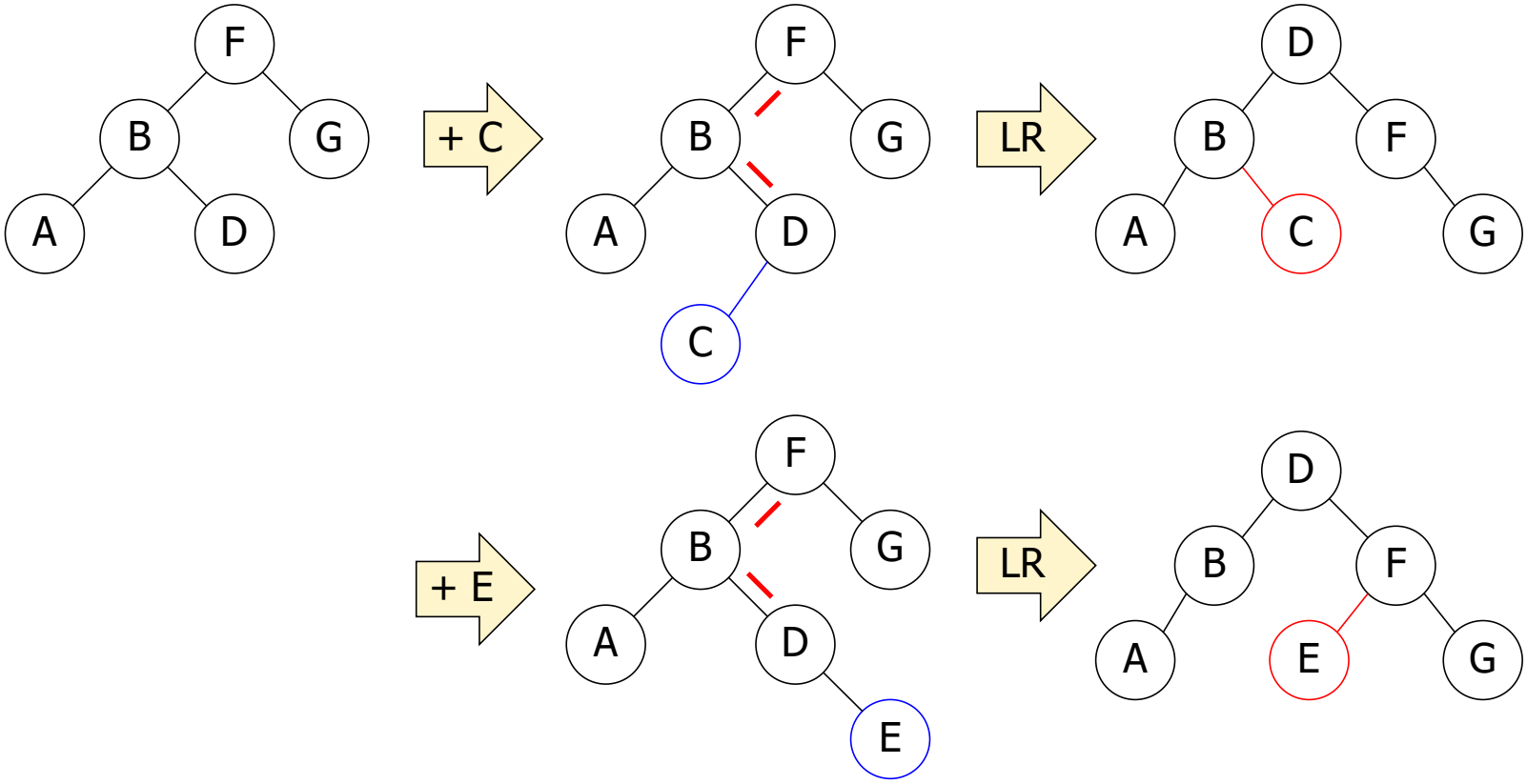
- Introduction
  - G. M. Adel'son-Vel'skii & E. M. Landis 에 의해 제안
  - Height-balanced tree – HB(1) tree

- 특징
  - 2 개의 Subtree 들간에 depth 차이 ≤ 1
  - 4 개의 회전 (LL, LR, RL, RR) 에 의해 tree 재구성
  - 포인터 변화는 5 개 이내로 국한
  - 예: B C G E F D A 순으로 입력될 때 (Page 58 그림)
  - Search cost : $1.44\log_2(N+2)$

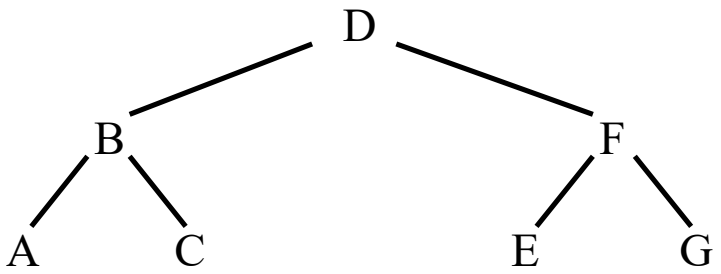
4.1 Tree 재구성



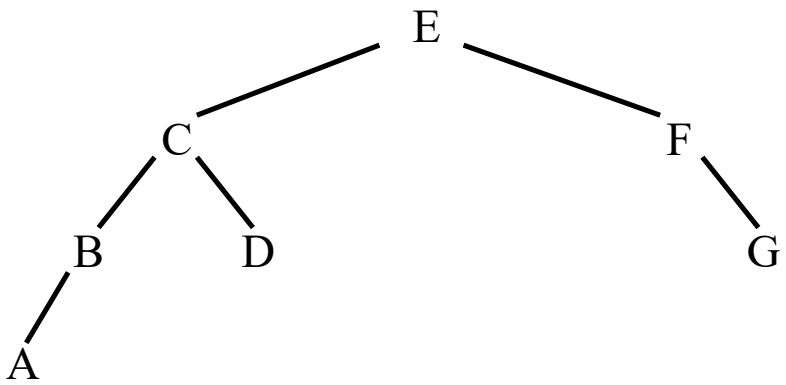
Tree 재구성(2)



Binary Search Tree vs. AVL Tree



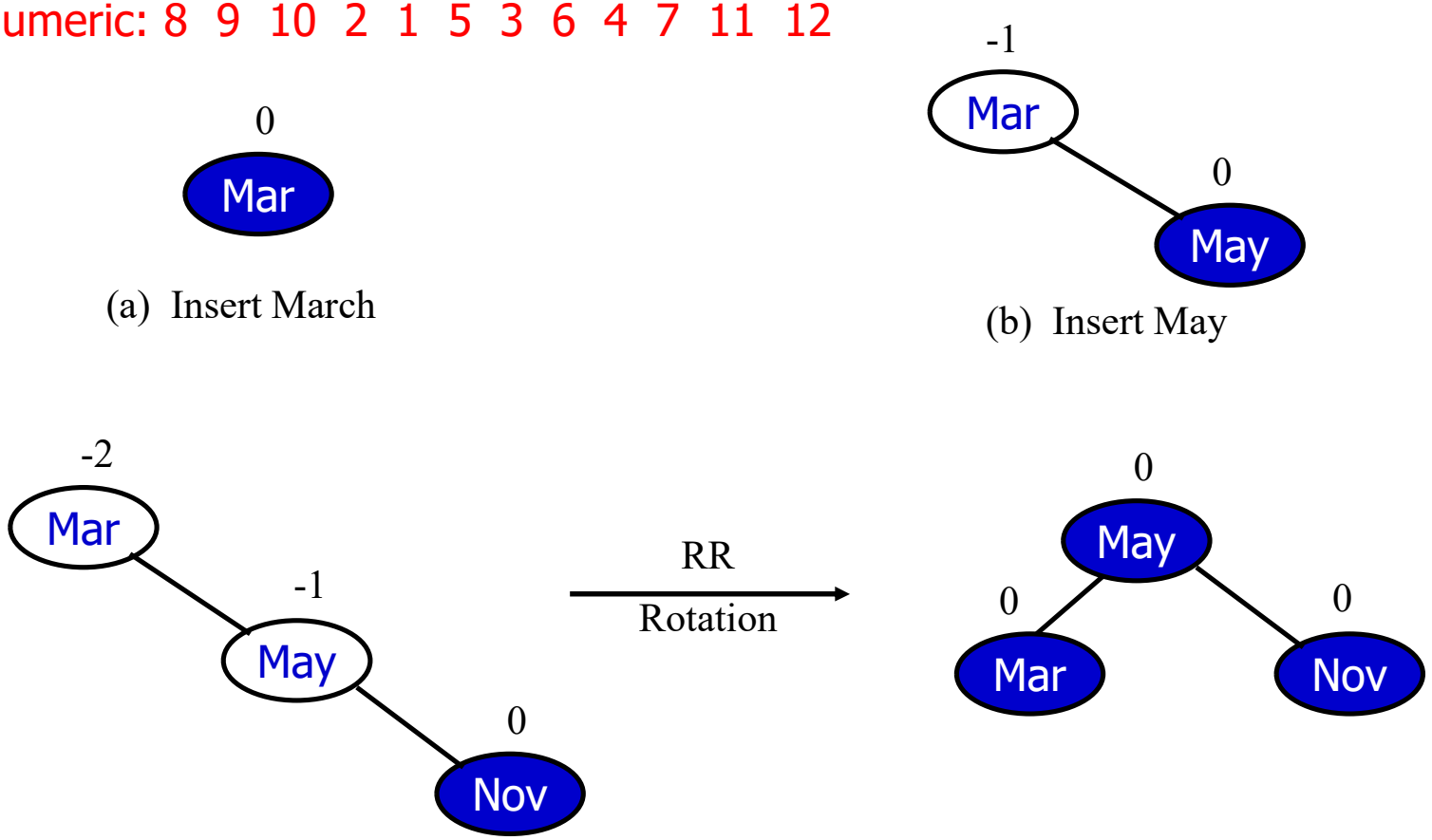
A completely balanced search tree.



A search tree constructed using AVL procedures.

4.2 Insertion into an AVL tree

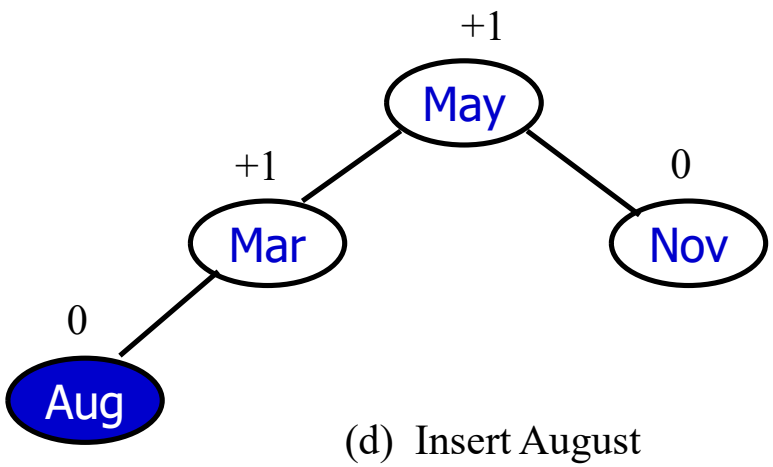
Numeric: 8 9 10 2 1 5 3 6 4 7 11 12



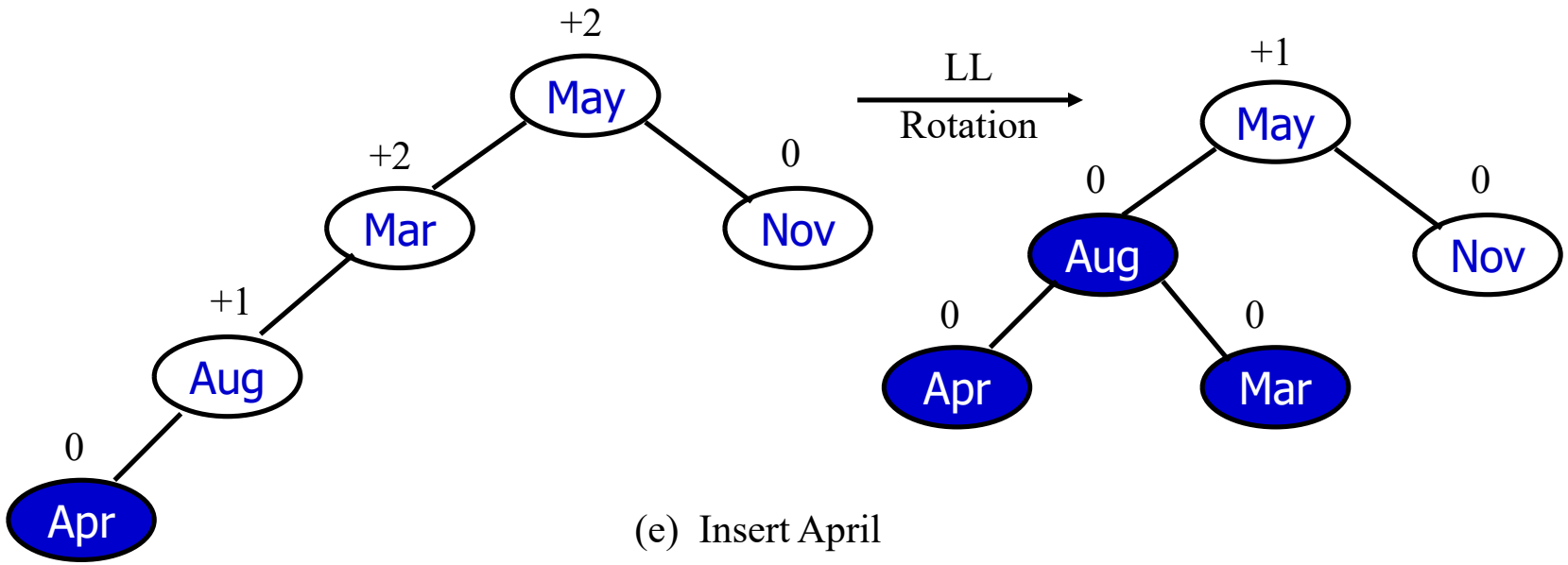
(a) Insert March

(b) Insert May

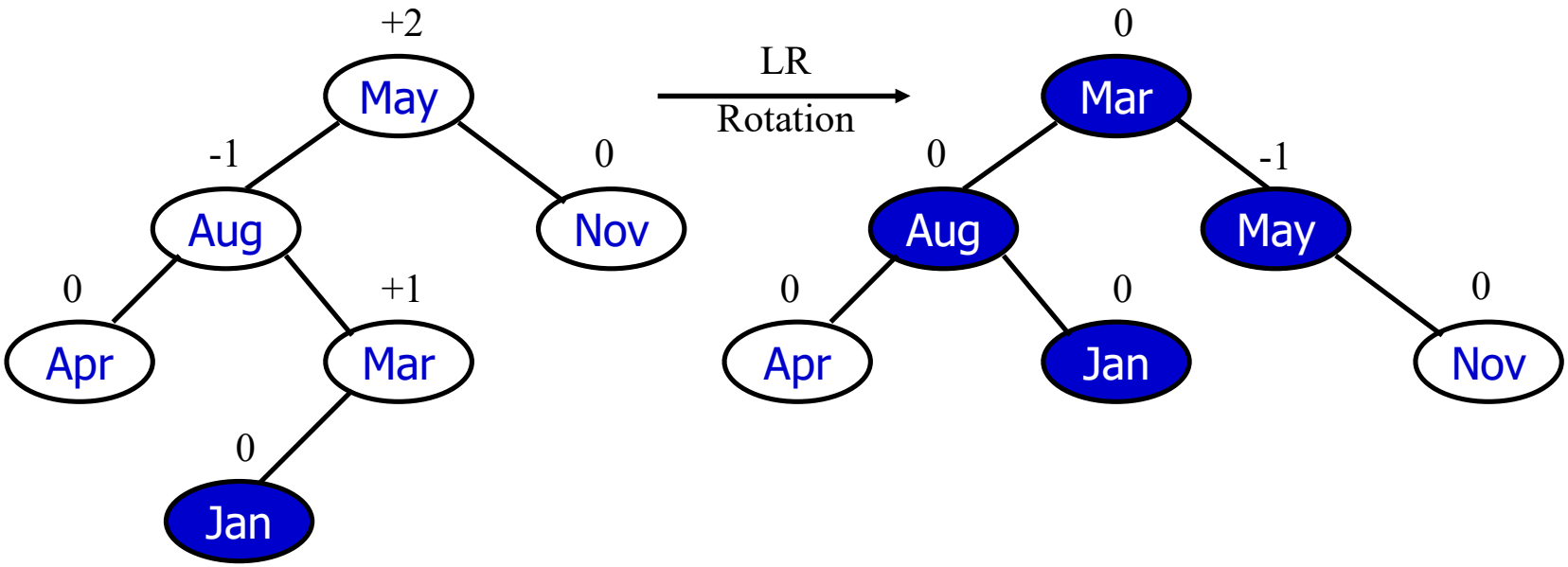
(c) Insert November



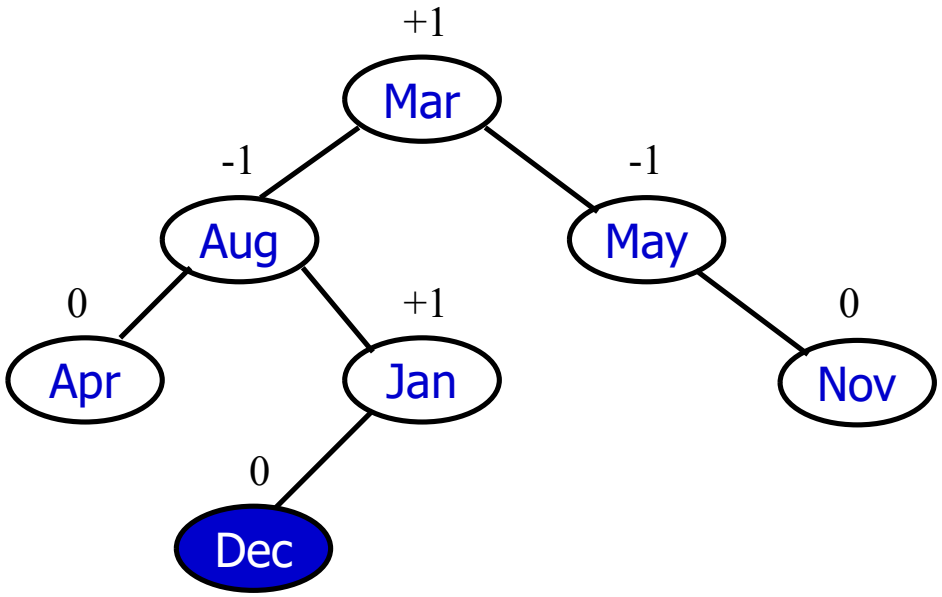
(d) Insert August



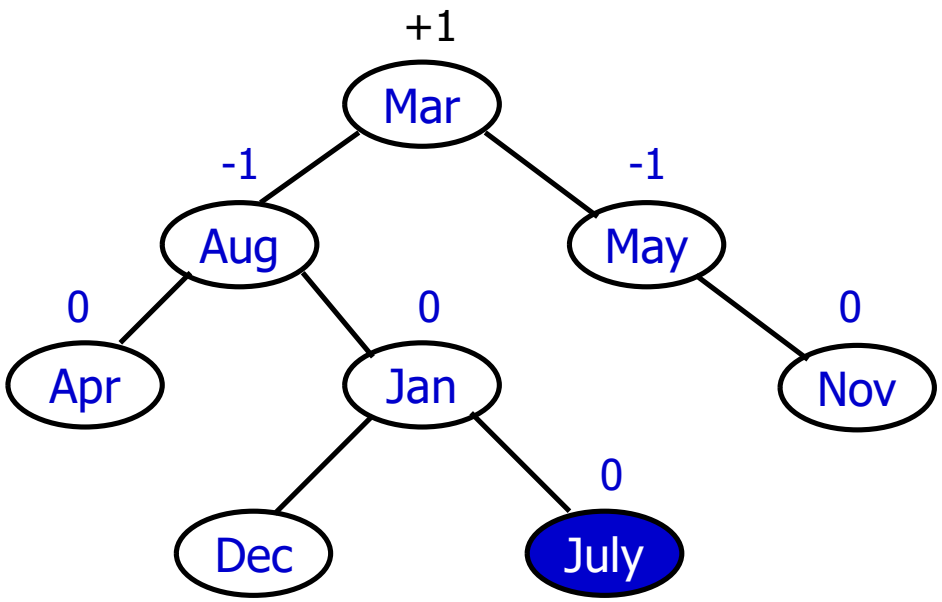
(e) Insert April



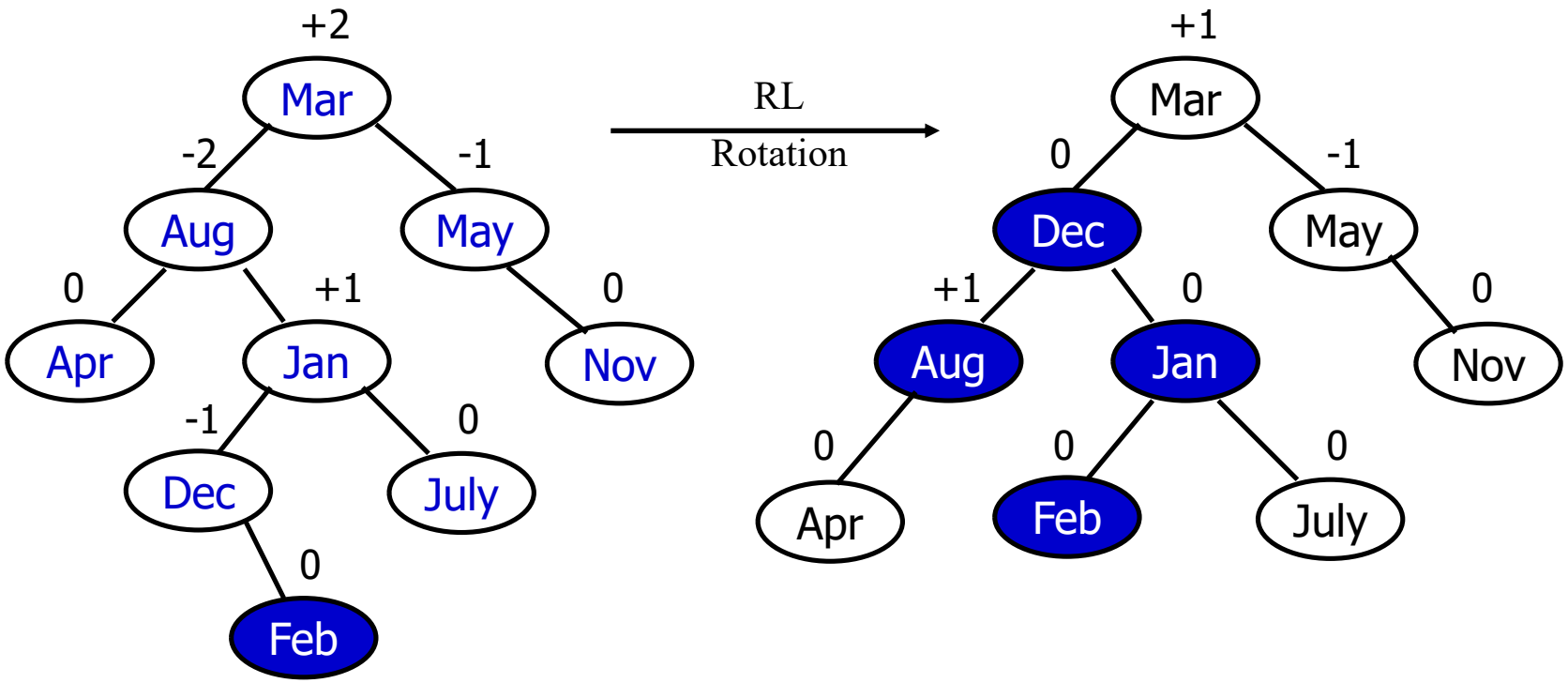
(f) Insert January



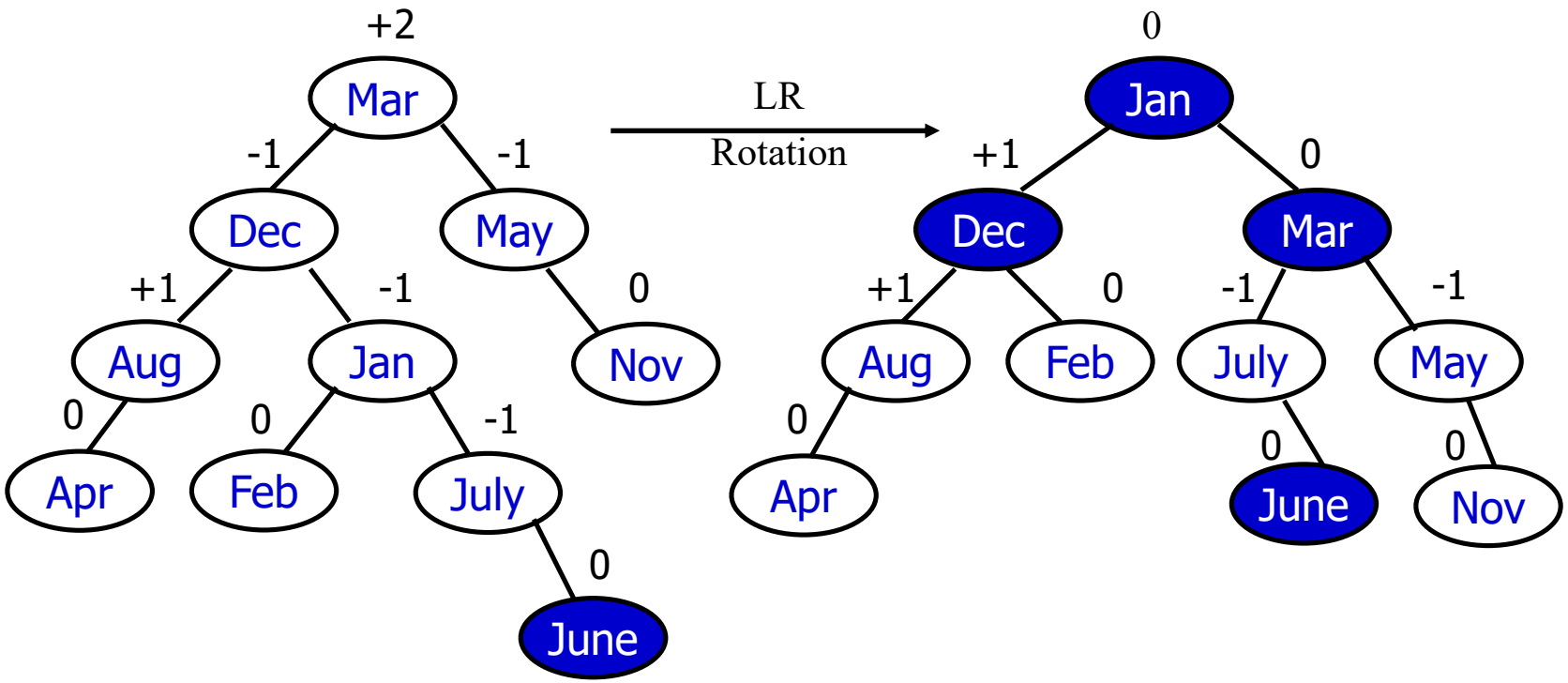
(g) Insert December



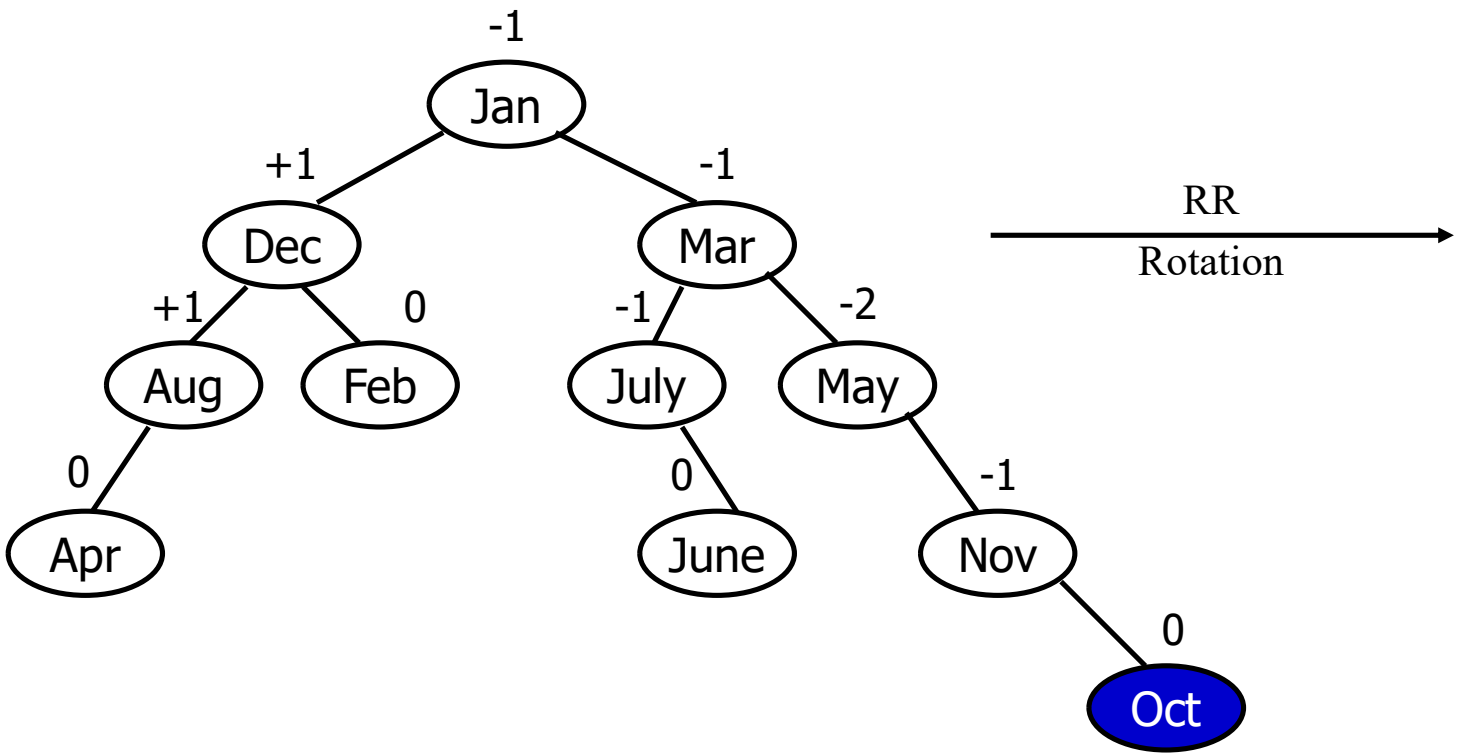
(h) Insert July



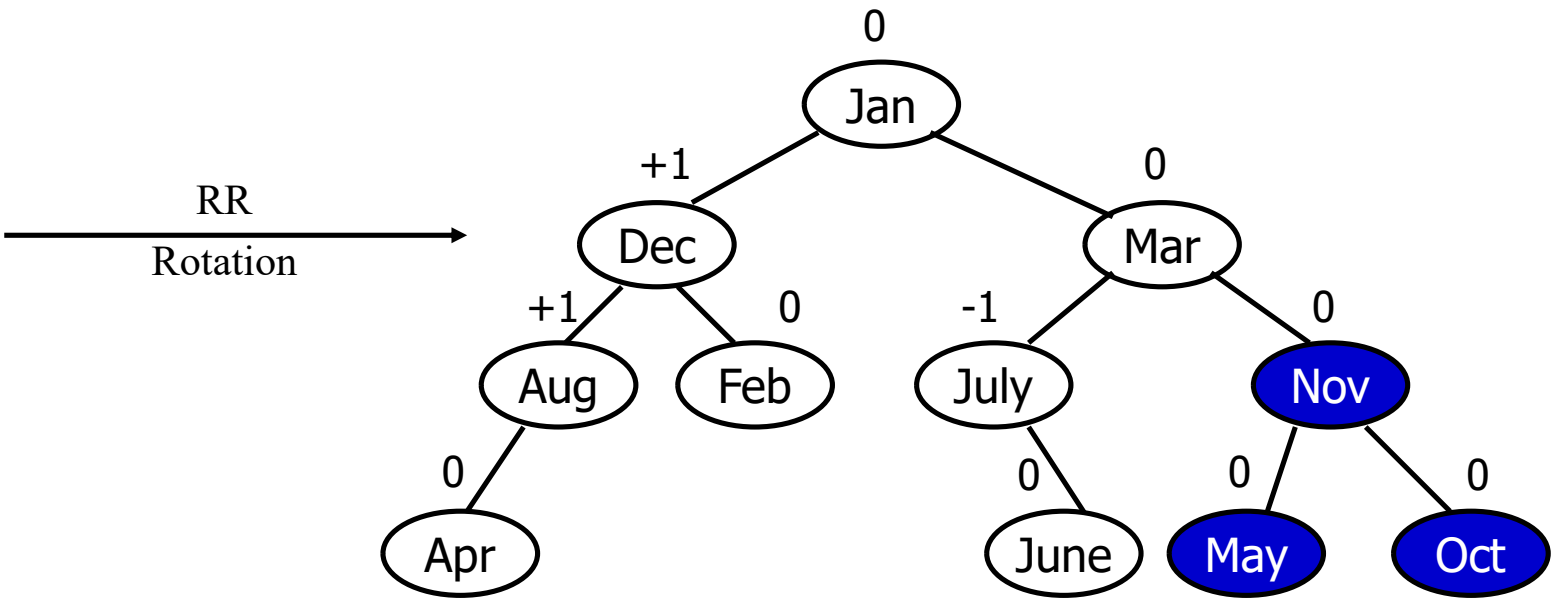
(i) Insert February



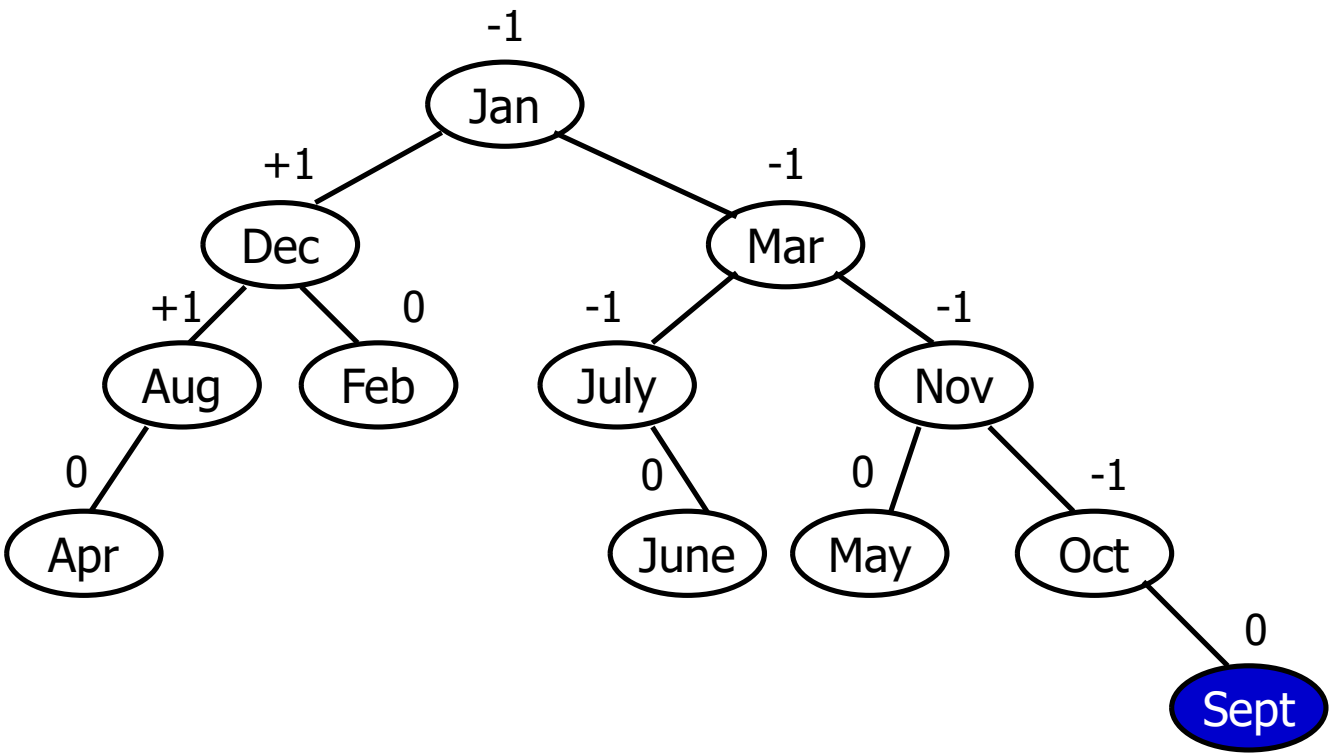
(j) Insert June



(k) Insert October



(k) Insert October



(l) Insert September

4.3 AVL Tree의 높이

- n(h): 높이가 h인 AVL 트리에 포함된 최소 노드 수
  - $n(1) = 1, n(2) = 2$
  - $n(h) = 1 + n(h-1) + n(h-2)$

  - $n(h-1) > n(h-2) \Rightarrow n(h) > 2n(h-2)$. 따라서,
  - $n(h) > 2n(h-2) > 4n(h-4) > 8n(h-6) > \ldots > 2^i * n(h-2i)$
  - $h-2i \geq 1$ 이므로, $h-2i = 2$로 두면 $i = \lceil h/2 \rceil - 1$
  - $n(h) > 2^i * n(h-2i) \geq 2^{\lceil h/2 \rceil - 1} * n(2) \geq 2^{\lceil h/2 \rceil}$
  - $n(h) > 2^{\lceil h/2 \rceil} \Rightarrow \log(n(h)) > h/2$
  - $h < 2\log(n(h))$

  - 따라서, AVL 트리의 높이: $O(\log N)$

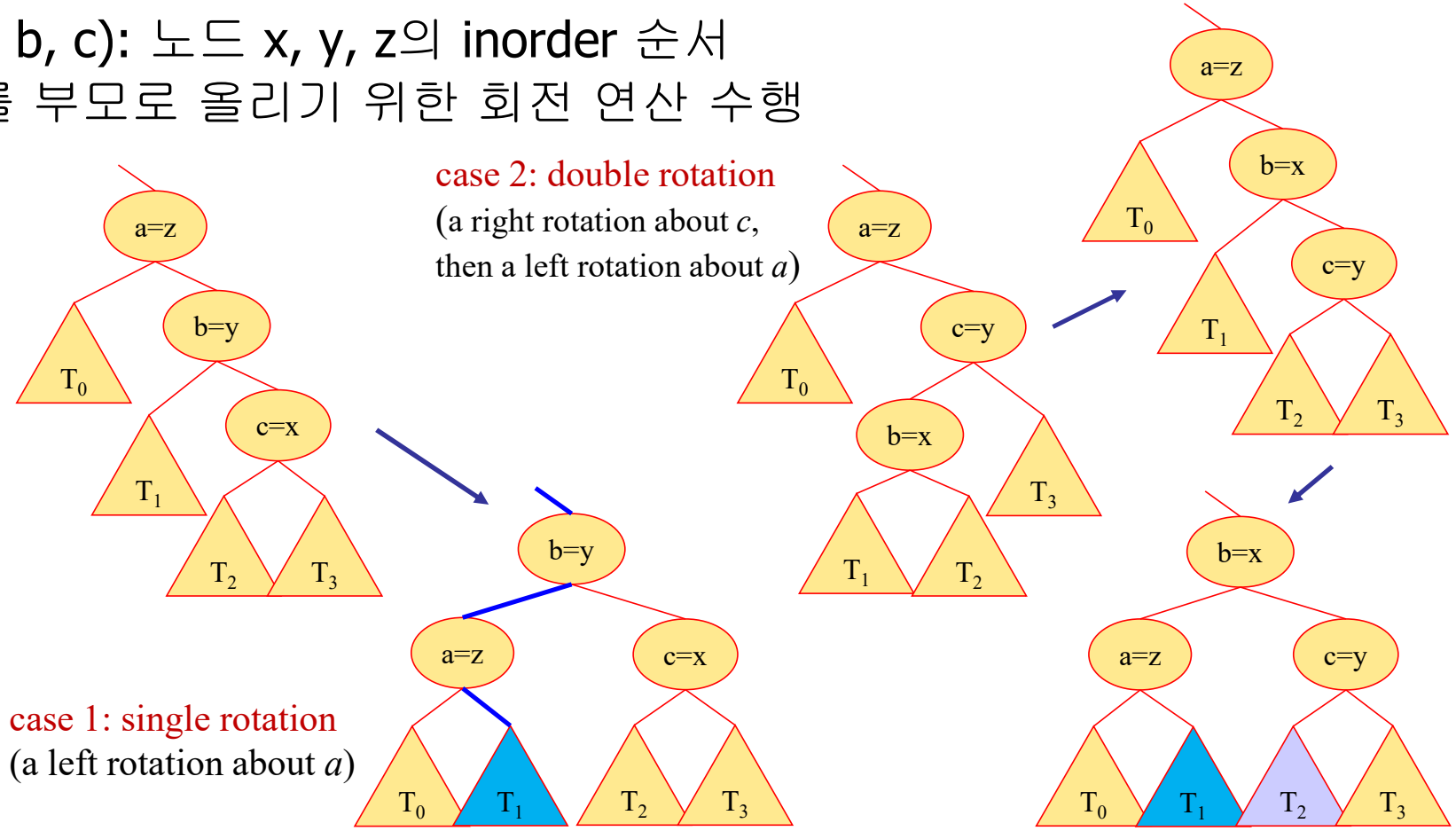
4.4 AVL Tree의 구현 – Java

- Node에 aux 필드 추가

```
public class Node<K,V> {
   K key;
   V value;
   int N;
   int aux;    // 노드의 높이: 리프 노드의 높이 = 1
   Node<K,V> left, right;
   Node<K,V> parent;   // 부모 노드의 포인터

   public Node(K key, V val) {
      this.key = key; this.value = val; this.N = 1;
   }

   public int getAux(){ return aux; }
   public void setAux(int value) { aux = value; }
}
```

Trinode Restructuring

- (a, b, c): 노드 x, y, z의 inorder 순서
- b를 부모로 올리기 위한 회전 연산 수행



```
protected void relink(Node<K,V> parent, Node<K,V> child, boolean makeLeft) {
    if (child != null)    child.parent = parent;        // child를 parent의 자식으로
    if (makeLeft)    parent.left = child;               // 왼쪽 자식, 또는
    else        parent.right = child;                   // 오른쪽 자식
}

protected void rotate(Node<K,V> x) {      // LL과 RR을 하나의 함수로. c → x → y가 문제!
    Node<K,V> y = x.parent;
    Node<K,V> z = y.parent;
    if (z == null) { root = x; x.parent = null; }
    else                relink(z, x, y == z.left);     // x를 z의 자식으로. 즉, x가 c와 y의 부모

    if (x == y.left)     {   relink(y, x.right, true); relink(x, y, false); }    // LL
    else                 {   relink(y, x.left, false); relink(x, y, true); }     // RR
}

protected Node<K,V> restructure(Node<K,V> x) {     // x → y → z가 문제
    Node<K,V> y = x.parent;
    Node<K,V> z = y.parent;
    if ((x == y.left) == (y == z.left)) {  rotate(y); return y; }     // LL or RR : y가 중간값
    else   {rotate(x); rotate(x); return x; }      // 중간값이 x: LR/RL → LL/RR로 일단 변경
}
```

AVL Tree를 이용한 심볼 테이블 구현

- BST의 하위 클래스로 구현
  - rebalanceInsert()와 rebalanceDelete() 함수만 다시 작성

- rebalanceInsert() ← put()의 마지막 연산
  - 리프 노드 x를 추가한 경우, 이 노드부터 루트 노드까지의 경로에 있는 모든 노드에 대해 height를 다시 계산
    - x.height = 1 + Math.max(height(x.left), height(x.right));
  - 경로의 노드에 대해 균형 여부를 조사
    - Math.abs(height(x.left) - height(x.right)) <= 1
  - 균형 노드가 아닌 노드에 대해 restructure() 함수 호출
    - x = restructure(tallerChild(tallerChild(x)));

- rebalanceDelete() ← delete()의 마지막 연산
  - 삭제한 노드 x와 x의 부모 노드 p를 입력받아 p부터 루트까지 height를 재조정한 후, rebalance(p) 실행

AVL Tree의 구현 – 기초 작업들

```
private int height(Node<K,V> x)   { return (x == null) ? 0 : x.getAux(); }
private void setHeight(Node<K,V> x, int height){ x.setAux(height); }

private void recomputeHeight(Node<K,V> x)     // 노드의 높이 재계산
{  setHeight(x, 1 + Math.max(height(x.left), height(x.right))); }

private boolean isBalanced(Node<K,V> x)        // 노드의 balanced 여부 검사
{  return Math.abs(height(x.left) - height(x.right)) <= 1;       }

private Node<K,V> tallerChild(Node<K,V> x) {  // 깊이가 깊은 자식 노드 조사
    if (height(x.left) > height(x.right)) return x.left;
    if (height(x.left) < height(x.right)) return x.right;
    if (x == root)              return x.left;          // 같은 경우에는 왼쪽 자식
    if (x == x.parent.left)     return x.left;          // LL이나
    else                        return x.right;         // RR을 선호
}
```

AVL Tree의 구현 – rebalance()

```
// rebalanceInsert()와 rebalanceDelete()에서 재구성을 위하여 호출하는 함수
private void rebalance(Node<K,V> x) {
    do {
        if (!isBalanced(x)) {   // x가 balanced node가 아닐 경우, 재구성
            x = restructure(tallerChild(tallerChild(x)));
            recomputeHeight(x.left);   // 재구성 후, 높이를 다시 조정
            recomputeHeight(x.right);
            for (Node<K,V> p = x; p != null; p = p.parent)
                recomputeHeight(p);
        }
        // put()의 경우에는 재구성 후, 부모 노드를 조사할 필요가 없으나,
        // delete의 경우에는 부모 노드들의 balanced 여부를 조사하여야 함.
        x = x.parent;
    } while (x != null);
}
```

FrequencyCounter의 실행 결과

	tinyTale.txt	tale.txt	leipzig1M.txt
순차 검색	6ms	1,054ms	INF
이진 검색	2ms	149ms	203,874ms
BST	0ms	124ms	10,951ms
AVL Tree	0ms	134ms	11,544ms

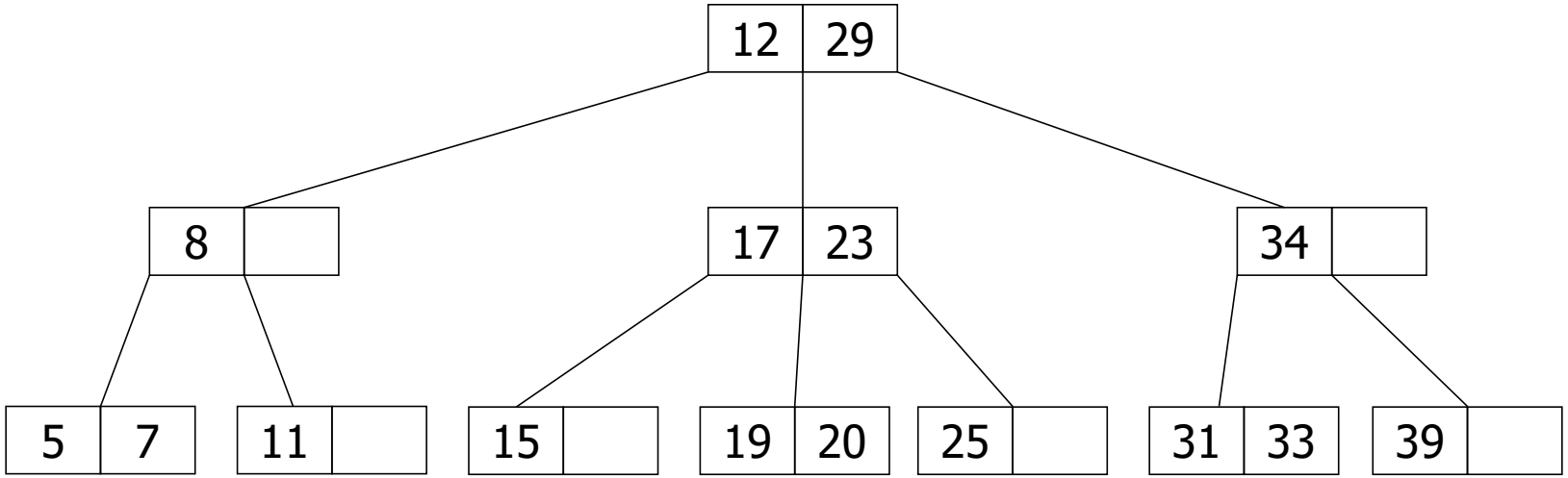
* 8자 미만의 단어들은 무시

5. Two-Three Trees (2-3 Trees)

- Definition
  - Internal node: 2-node or 3-node
    - 2-node: one element with two children
    - 3-node: two element with three children
  - All external nodes are at the same level.
    - 모든 리프 노드들은 동일한 레벨

- Properties
  - n: number of elements, h: height of 2-3 trees
  - $2^h - 1 \le n \le 3^h - 1$
  - $\lceil \log_3(n + 1) \rceil \le h \le \lceil \log_2(n + 1) \rceil$

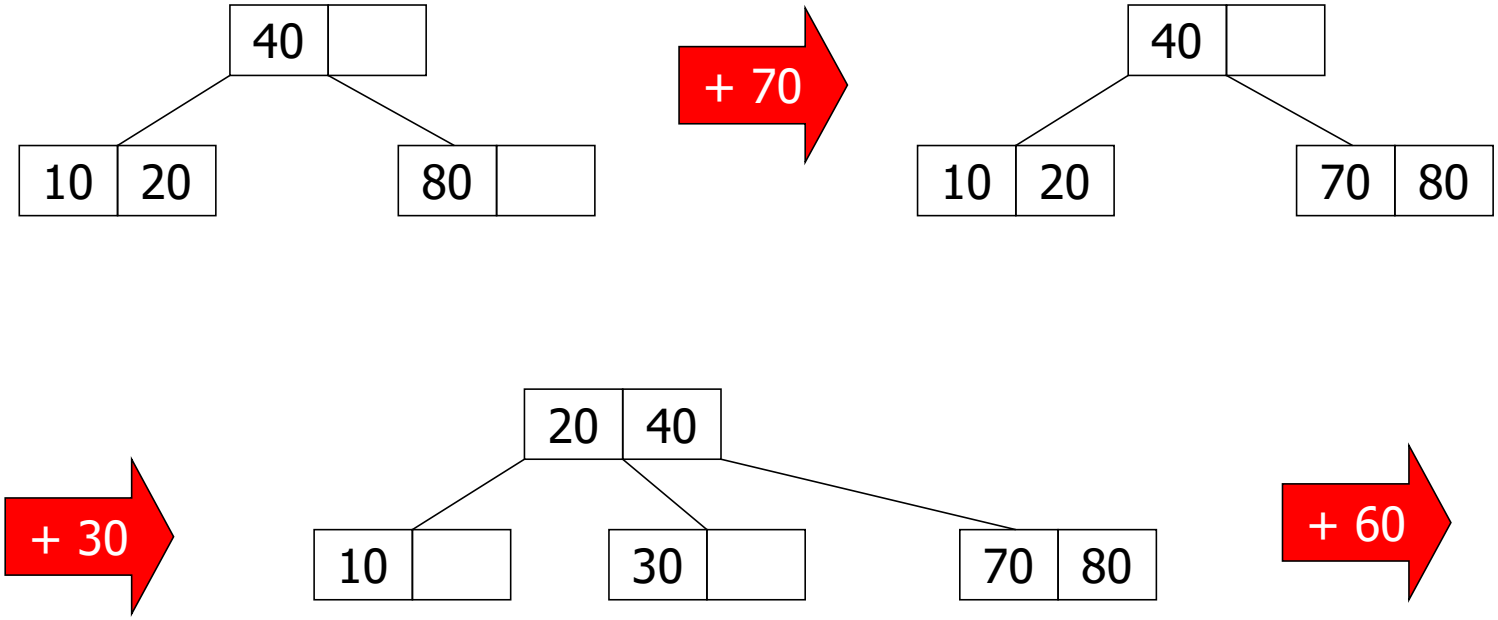
2-3 Tree의 예

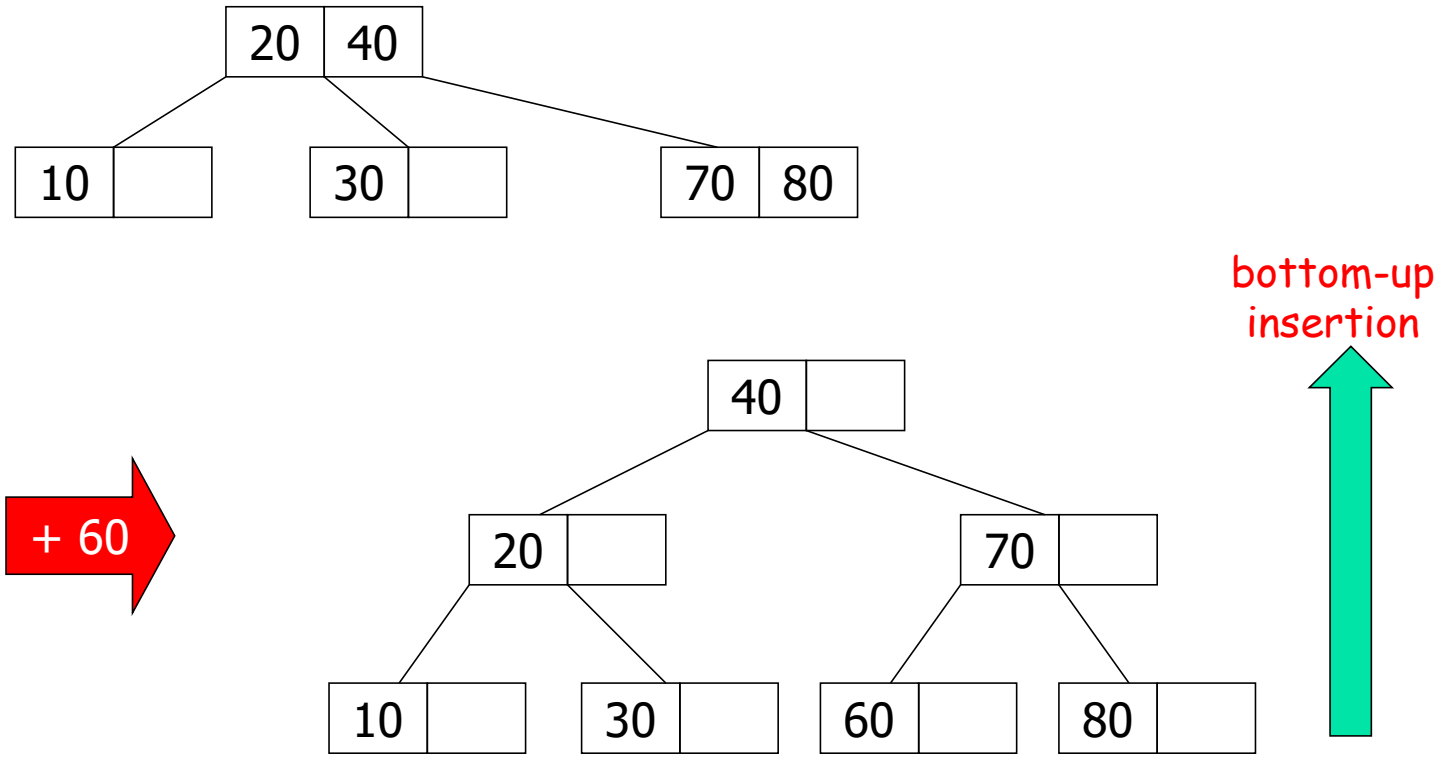


Searching a 2-3 Tree

```
typedef struct two_three *two_three_ptr;
struct two_three {
   element          data_l, data_r;
   two_three_ptr    left_child, middle_child, right_child;
};

two_three_ptr search23(two_three_ptr t, element x)
{
   while (t)
      switch (compare(x, t)) {
         case 1:   t = t→left_child;      break;   // x < t→data_l
         case 2:   t = t→middle_child;    break;   // t→data_l < x < t→data_r
         case 3:   t = t→right_child;     break;   // x > t→data_r
         case 4: return t;
      }
   return NULL;
}
```

Insertion Into a 2-3 Tree




20
40
10
30
70
80
+ 60
40
20
70
10
30
60
80
bottom-up
insertion

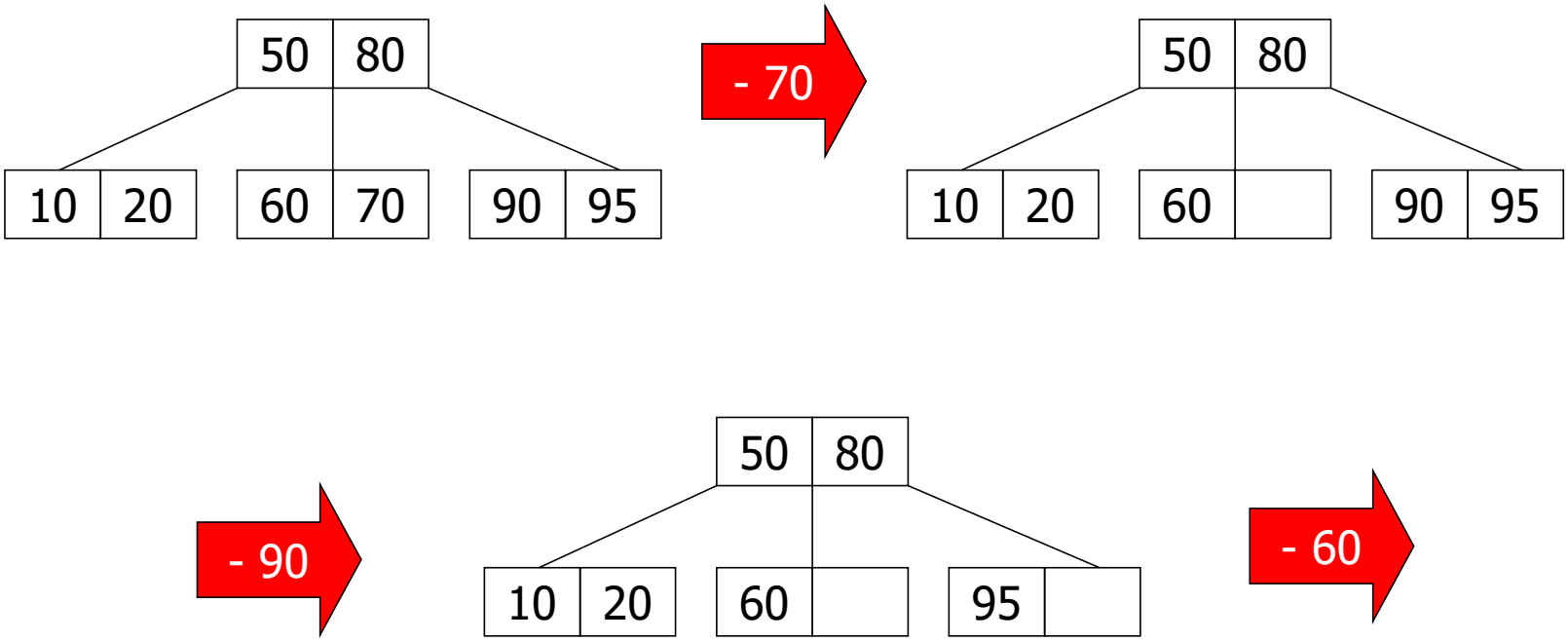
Insertion into a 2-3 tree

```
void insert23 (two_three_ptr *t, element y)
{
    two_three_ptr          q = NULL, p;

    if (!(*t))  new_root(t, y, NULL);
    else {
          if (!(p = find_node(*t, y)))        exit(1);   // duplicate
          while (1)
                    if (p→data_r.key == INT_MAX) // 2-node
                    {         put_in(&p, y, q);  break;          }
                    else {    // 3-node
                              split(p, &y, &q);
                              if (p == *t) {    // split the root
                                        new_root(t, y, q);  break;
                              }
                              else      p = delete();
}   }               }
```

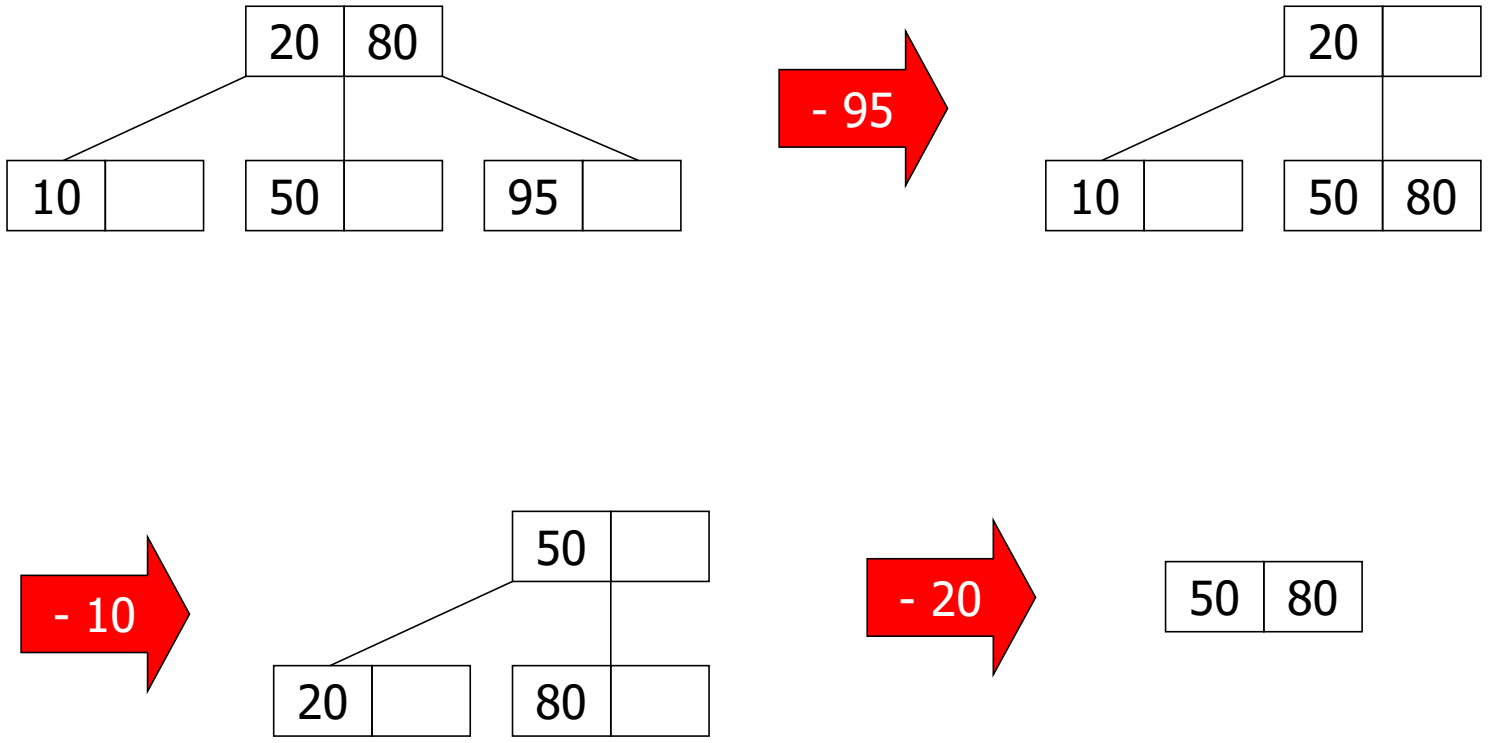
Deletion From a 2-3 Tree

- Leaf node가 아닌 키 값 삭제: leaf node 삭제로 변경
  - Inorder predecessor와 변경, or
  - Inorder successor와 변경
- Leaf node의 삭제 방법
  - 3-node: 단순 삭제, Node 내에서 키 값 이동 필요
  - 2-node: Node들간의 키 값 이동 필요
    - Rotation
    - Combine

Example – 단순 삭제



Example: Rotate & Combine



Steps in deleting from a leaf node, *p*

Step 1: Modify *p* as necessary to reflect its status after the desired element has been deleted.

Step 2: while (*p* has zero elements && *p* is not the root) {

```
        let r be the parent of p;
        let q be the left or right sibling of p (as appropriate);
        if (q is a 3-node)          rotate;
        else                        combine;
        p = r;
}
```

Step 3: If *p* has zero elements, then *p* must be the root. The left child of *p* becomes the new root and *p* is deleted.

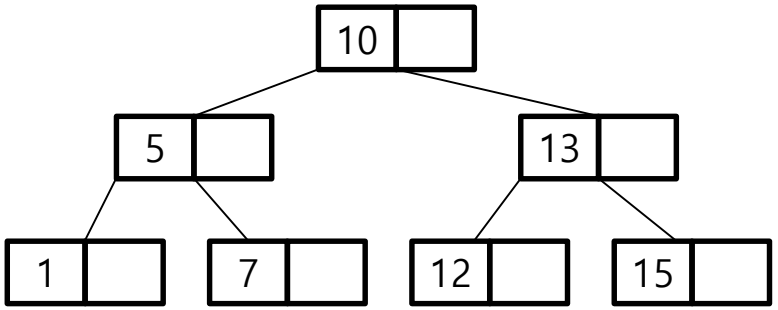
연습 문제(1)

- 아래 키 값이 입력될 때 생성되는 2-3 트리를 그리시오.

8 → 9 → 10 → 1 → 2 → 7 → 6 → 5 → 4

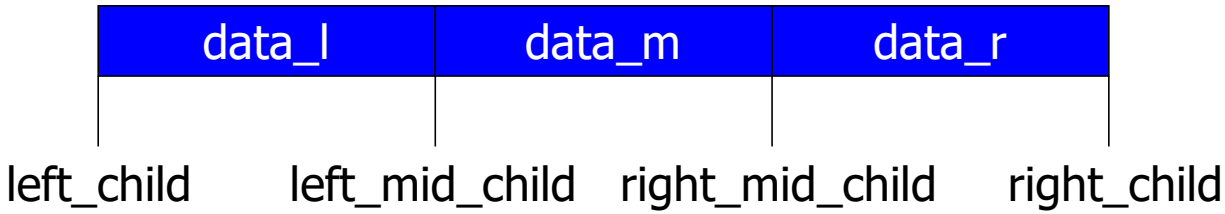
연습 문제(2)

- 아래 2-3 트리에서 10이 삭제되고 난 후의 2-3 트리를 그리시오.

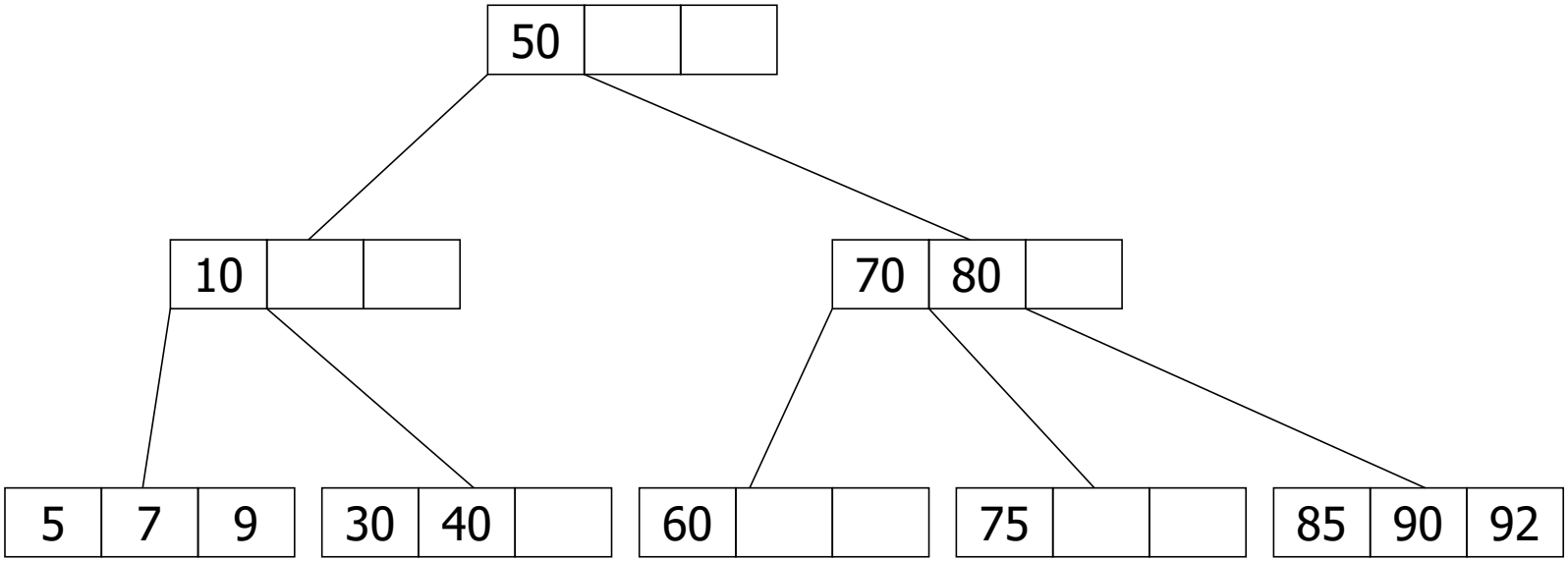


6. Two-Three-Four Trees (2-3-4 Trees)

- Definition: Definition of 2-3 Tree +
  - 4-node: three elements and four children
  - n: number of elements, h: height of 2-3-4 trees
  - $2^h - 1 \le n \le 4^h - 1$
  - $\lceil \log_4(n + 1) \rceil \le h \le \lceil \log_2(n + 1) \rceil$
- Node Structure



An Example 2-3-4 Tree

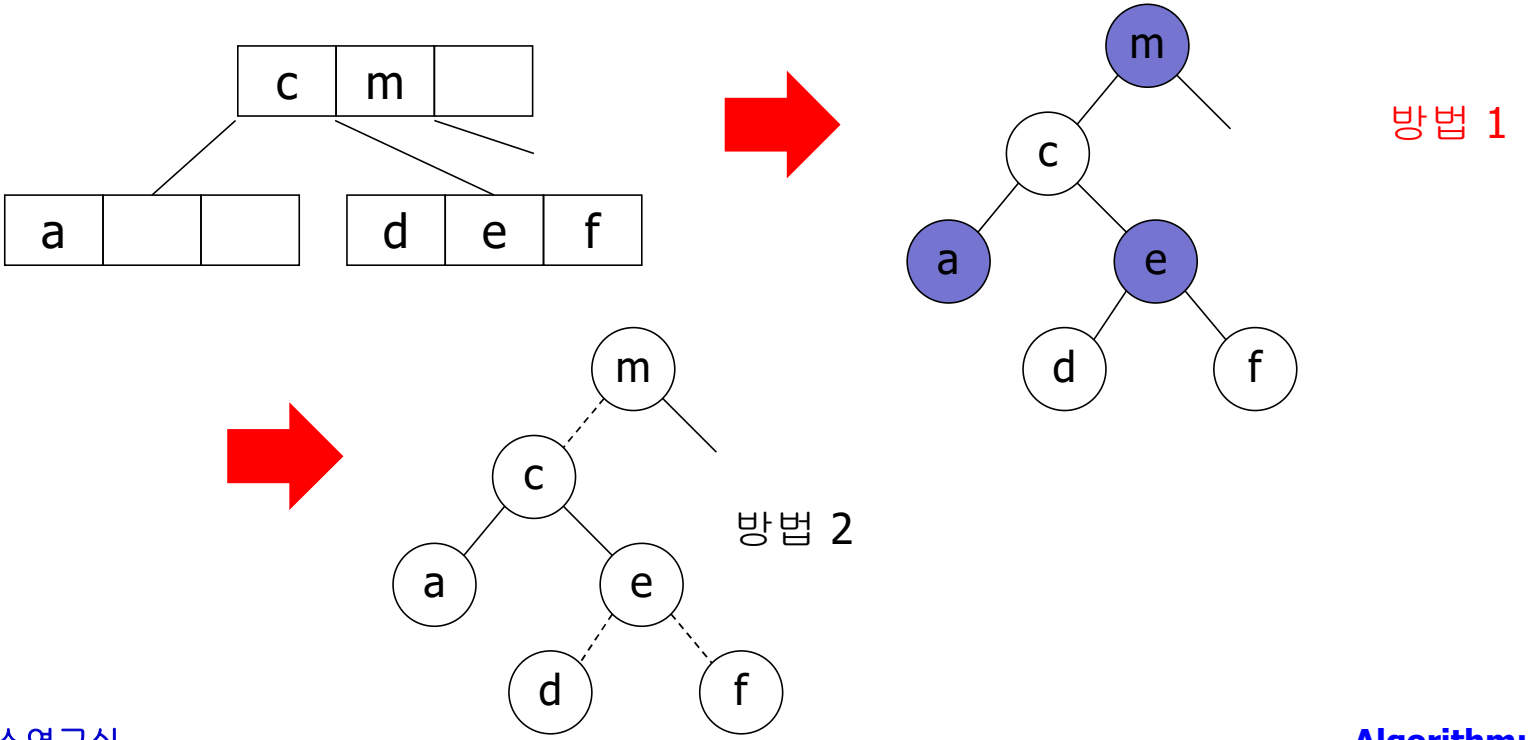


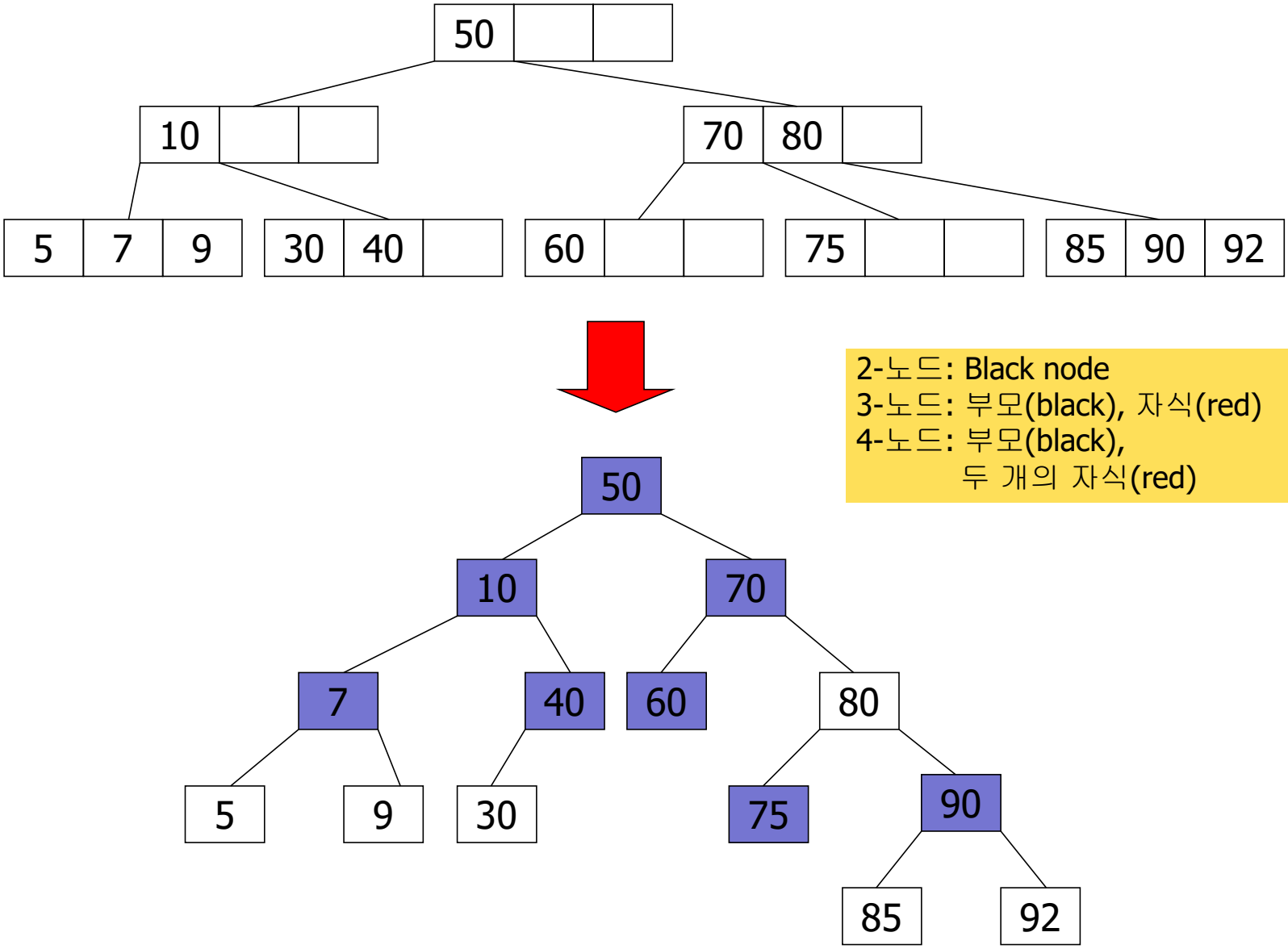
2-3-4 트리의 삽입과 삭제

- 2-3 트리의 경우와 거의 동일
- 삽입
  - Leaf 노드에 추가
  - Leaf 노드가 overflow: (a, b, c) + d
    - 노드 분할: (a, b) (d) & c는 부모 노드로
- 삭제
  - Non-leaf 노드의 삭제는 leaf 노드의 삭제로 변경
    - Inorder successor의 키와 변경
  - Leaf 노드의 삭제
    - 3-node, 4-node: 노드 내에서 해결
    - 2-node: Rotate or Combine

7. Red-Black Trees (RB Tree)

- Red-Black Tree: 2-3-4 tree를 이진 트리로 표현
  - 방법 1: 노드를 Black node와 Red node로 구분
  - 방법 2: 링크를 Black link과 Red link로 구분





2-노드: Black node
3-노드: 부모(black), 자식(red)
4-노드: 부모(black),
두 개의 자식(red)

Algorithm: Chapter 2 (Page 93)

7.1 RB Tree의 성질

- 기본적인 성질
  - Root property: 루트 노드는 black
  - Red property: Red node의 자식 노드는 black
    - 부모-자식이 동시에 red node가 될 수 없음
  - Depth property: 리프 노드에서 루트 노드까지의 모든 경로는 동일한 수의 black node로 구성

- n 개의 노드로 구성된 RB Tree의 높이 $h_{RB} = O(\log n)$
  - n개의 노드를 갖는 2-3-4 트리의 높이: $h_{234} \leq \log(n + 1)$
  - $h_{234} <= h_{RB} <= 2*h_{234}$
  - $h_{RB} <= 2*\log(n + 1)$

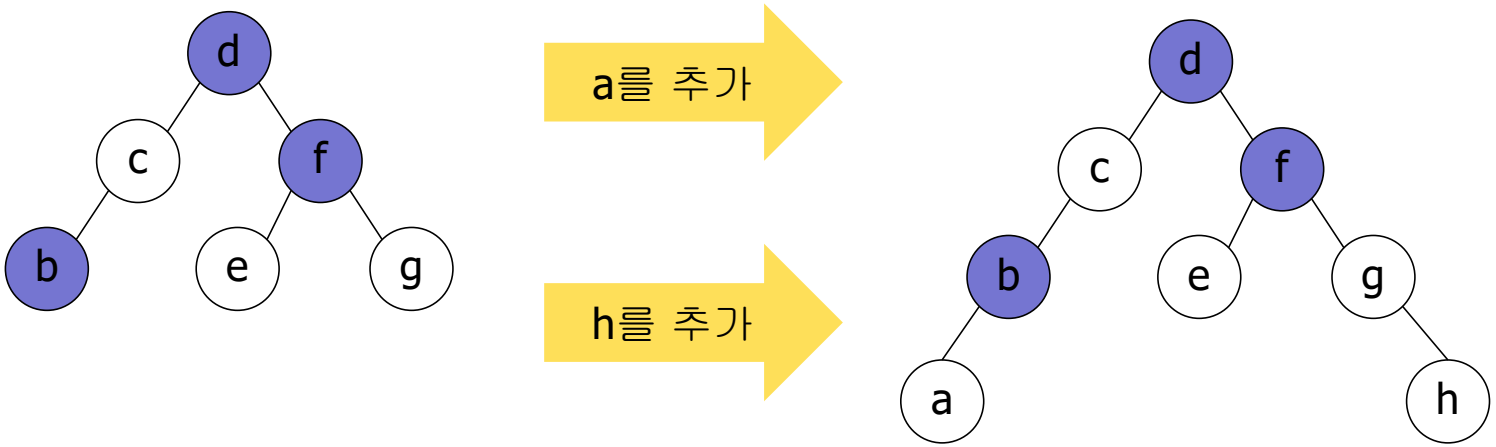
7.2 RB Tree의 연산

- Searching
  - 일반적인 BST의 검색 방법을 이용하여 검색
  - Node color는 사용되지 않음
- Insertion
  - BST의 삽입 과정과 유사
  - 노드 삽입 후 color 조정 (또는 회전) 필요
- Deletion
  - 2-3-4 tree의 삭제 알고리즘을 red-black tree에 적용

- RB Tree의 장점
  - AVL Tree나 2-3-4 트리에 비해 트리 재구성의 빈도수가 낮음
  - 트리 재구성 → Color 변경으로 대체 가능

Insertion

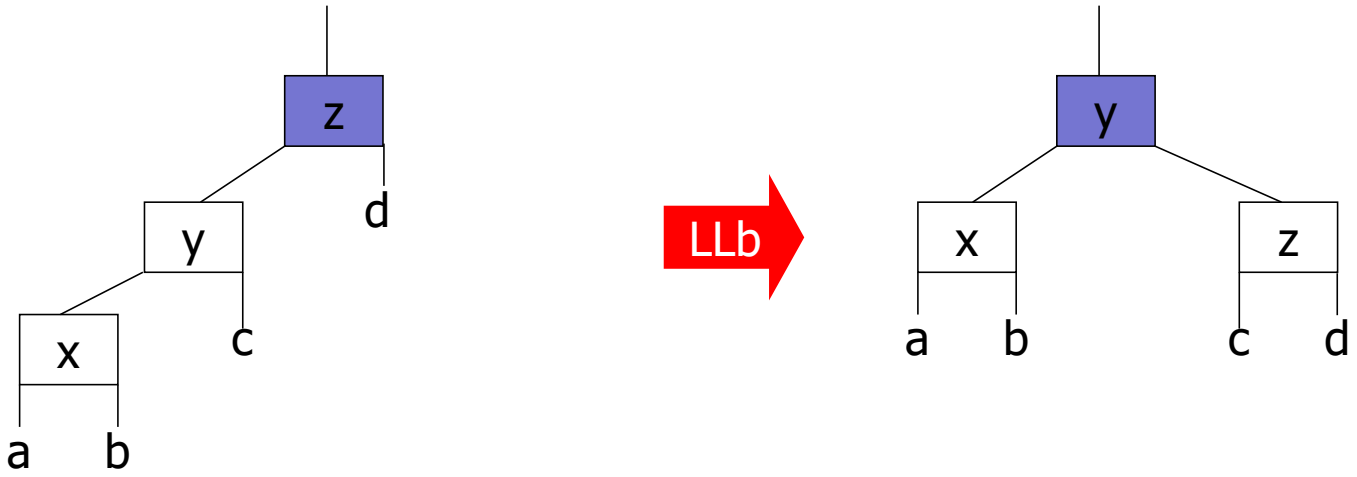
- 기본 규칙
  - BST의 put() 알고리즘에 의해 추가할 노드 z의 위치 결정
  - z가 루트이면, black
  - z가 루트가 아니면, red
    - z의 부모가 black일 경우에는 아무 문제 없음.
    - z의 부모가 red일 경우, double red 문제 발생
- 예

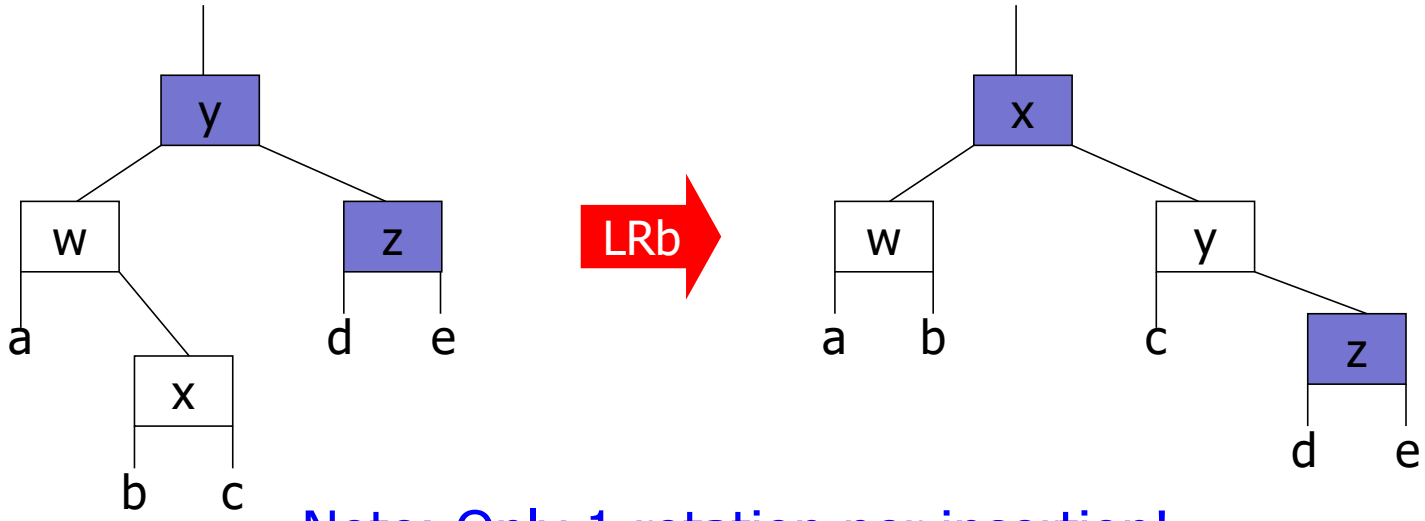


Double Red를 해결하는 방법

- 연속된 red node: z → y → x
  - s를 y의 sibling이라 할 때, ABC violation 정의
    - A = L, if z → y is a left link. A = R otherwise.
    - B = L, if y → x is a left link. B = R otherwise.
    - C = r (for red) if s ≠ NULL and s is a red node.
      C = b (for black) otherwise.
  - C = b: 4-node를 잘못 표현 → 재구성(회전) 필요
  - C = r: overflow → Color 변경(4-node의 분할)
    - Bottom up으로 color change 과정을 전파

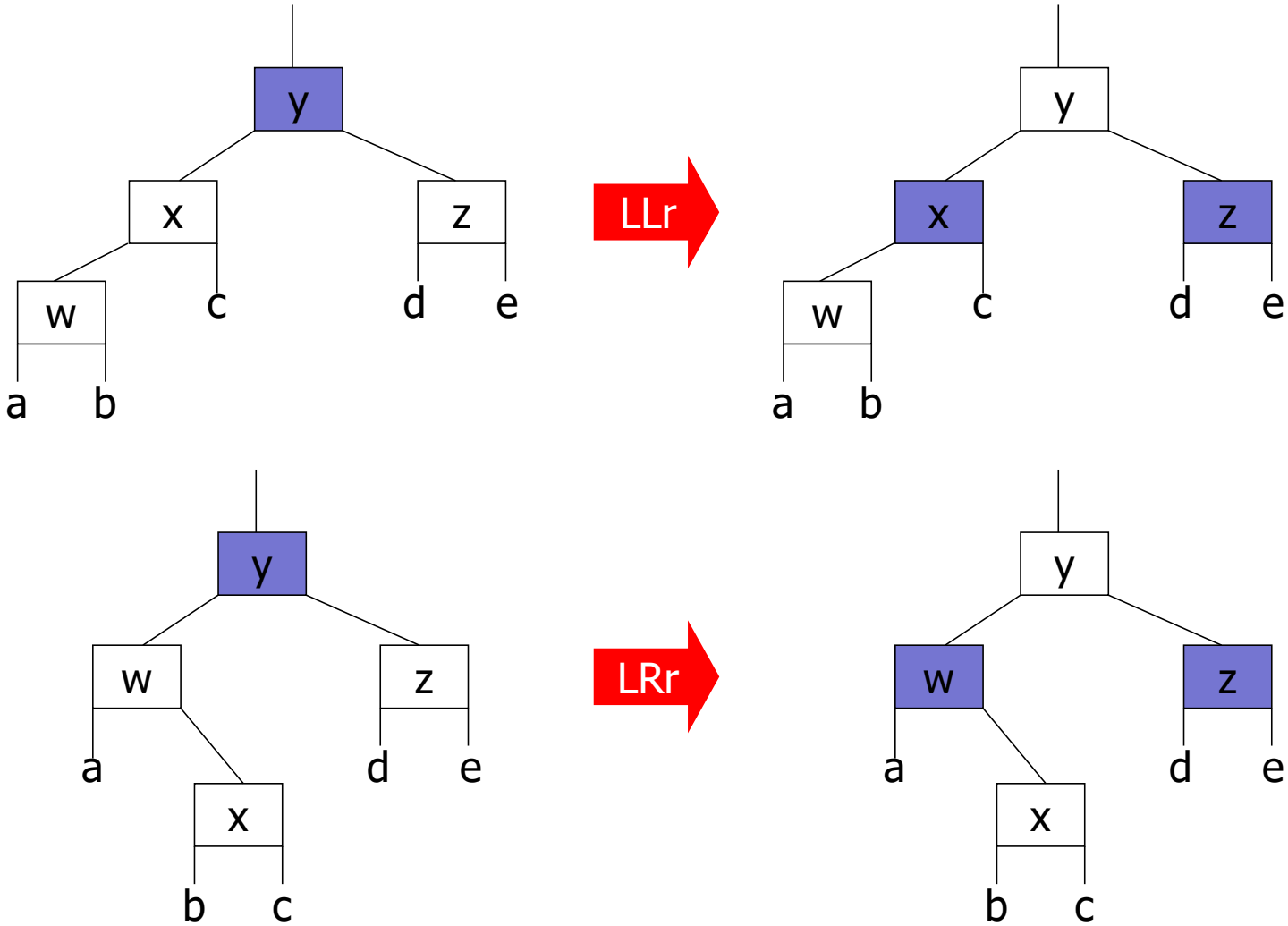
- LLr과 LRr, LLb와 LRb에 대한 변경 과정의 예





Note: Only 1 rotation per insertion!

Algorithm: Chapter 2 (Page 98)

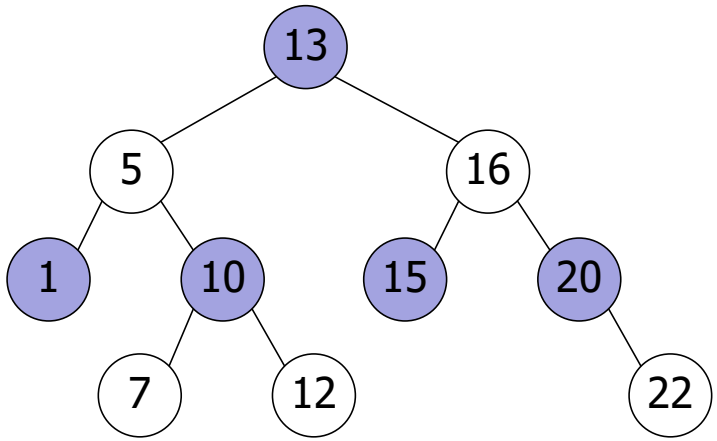


Algorithm: Chapter 2 (Page 99)

Example

- 아래 키 값이 입력될 때 생성되는 Red-Black 트리를 그리시오.

  1 → 5 → 7 → 13 → 15 → 10 → 12 → 20 → 16 → 22



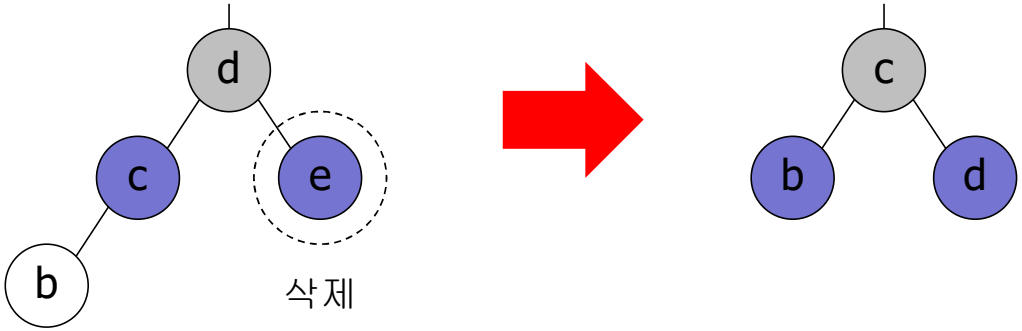
Deletion From a Red-Black Tree

- BST의 delete() 과정 수행
  - 단, 노드 삭제의 결과로 red property나 depth property가 만족되지 않을 수 있음

- 삭제할 노드 x의 color? (x는 2-3-4 트리에서 리프에 위치)
  - Red: x는 3-/4- node의 일부 (단순 삭제)
  - Black: x가 자식을 가지고 있는지 여부 확인
    - Red 노드를 자식으로 가지고 있을 경우: 자식을 x의 위치로 & 자식의 color를 Black으로.
    - 자식이 없을 경우: x는 2 node! ← underflow에 해당
      - 3가지 경우를 추가로 고려하여야 함.
      - 다음 slide 참조

Underflow의 해결 방법(1)

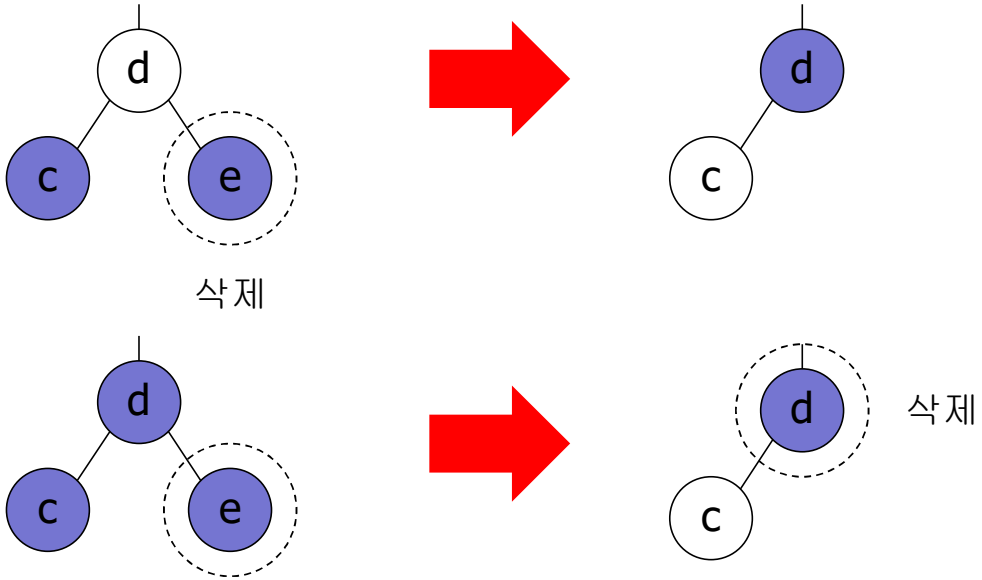
- 경우 1: Sibling이 black이며 red child를 가질 경우



- Tri-node restructuring (회전) 수행
  - 자식 노드들은 black으로
  - 부모 노드는 원래 부모 노드의 color로.
  - 이후 종료 가능.

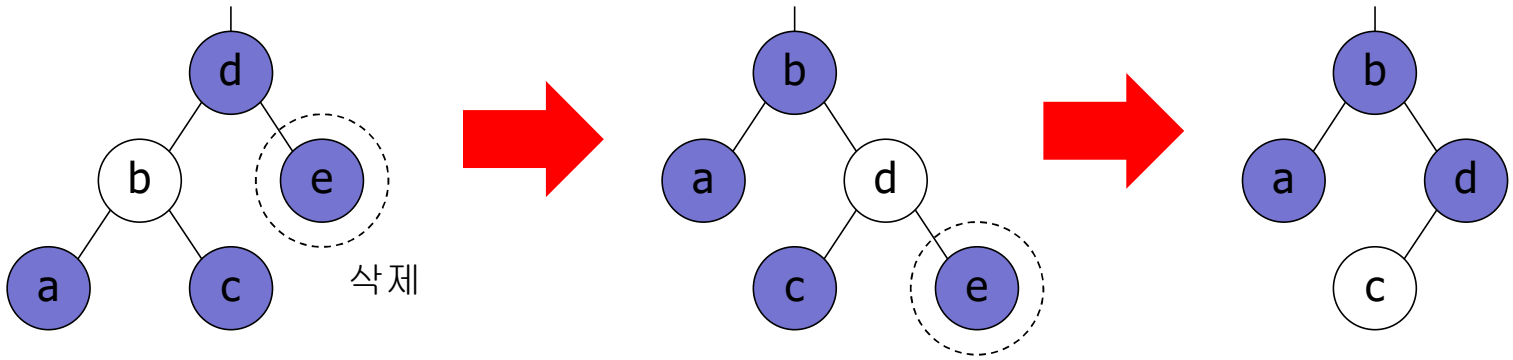
Underflow의 해결 방법(2)

- 경우 2: Sibling이 black이며 자식이 없거나, black child를 가질 경우
  - 부모가 red: 부모와 sibling을 3-node로 묶고, 종료
  - 부모도 black: 부모와 sibling을 3-node로 묶고, cascading delete

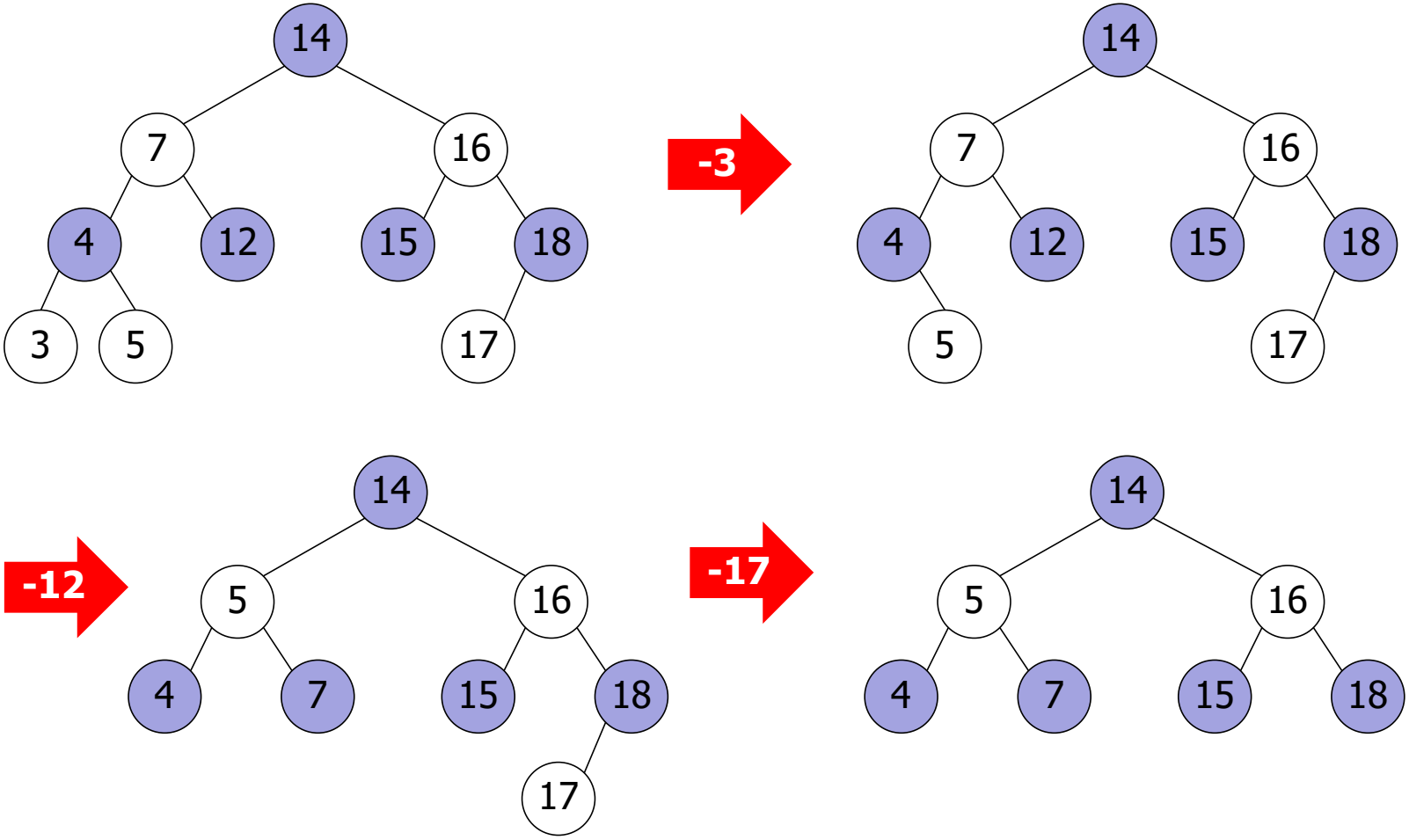


Underflow의 해결 방법(3)

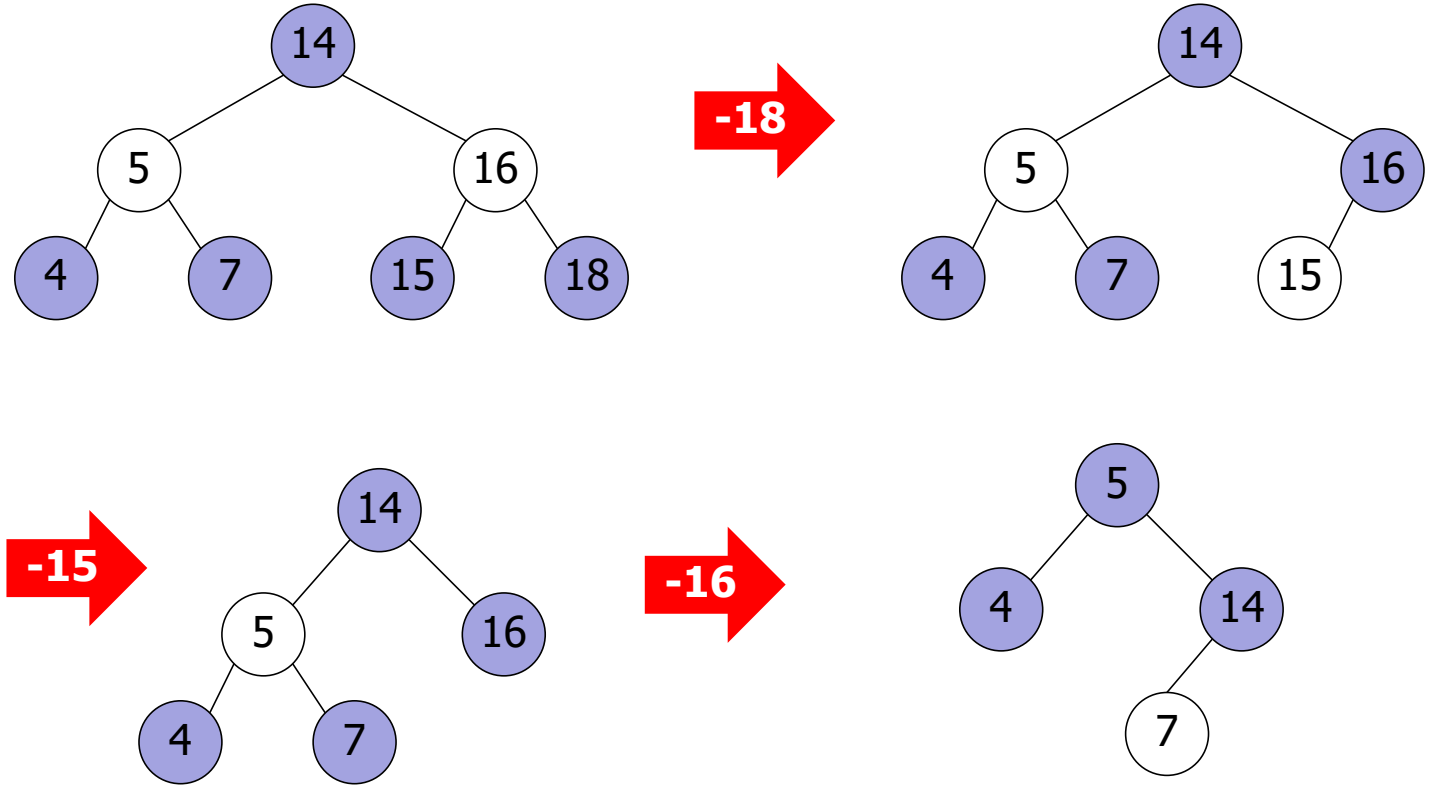
- 경우 3: Sibling이 red일 경우
  - 부모 노드가 3-node일 경우에 해당
  - 3-node의 다른 방식으로 부모 노드를 변경한 후,
  - 새로운 sibling에 대해 경우1 또는 경우 2 수행.
    - 경우 1은 한번의 연산으로 종료
    - 경우 2도 부모가 red이므로 바로 종료



Example (1)

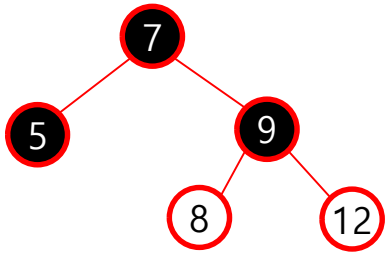

14
7
16
4
12
15
18
3
5
17
-3
14
7
16
4
12
15
18
5
17
-12
14
5
16
4
7
15
18
17
-17
14
5
16
4
7
15
18

Example (2)



연습 문제

- 아래와 같은 red-black 트리에서 물음에 답하라.



① 11과 10이 차례대로 입력된 후의 red-back 트리를 그리시오.
② ①의 트리에서 5가 삭제되고 난 후의 red-back 트리를 그리시오.

7.3 RB Tree의 구현

- BST의 하위 클래스로 구현
  - rebalanceInsert()와 rebalanceDelete() 함수만 다시 작성

```
boolean isBlack(Node<K,V> x)  { return (x == null) || (x.getAux() == 0); }
boolean isRed(Node<K,V> x)    { return (x != null) && (x.getAux() == 1); }
void makeBlack(Node<K,V> x)  { x.setAux(0); }
void makeRed(Node<K,V> x)    { x.setAux(1); }

Node<K,V> sibling(Node<K,V> x) {
    Node<K,V> p = x.parent;
    if (p.left == x)      return p.right;
    else                  return p.left;
}
```

RB Tree의 구현 – rebalanceInsert()

```
private void resolveRed(Node<K,V> x) {
    Node<K,V> parent, uncle, middle, grand;
    parent = x.parent;
    if (isRed(parent)) {    // x가 root가 아니므로, red → double-red problem
        uncle = sibling(parent);
        if (uncle == null || isBlack(uncle)) {      // XXb → 4-노드를 재구성
            middle = restructure(x);
            makeBlack(middle);
            makeRed(middle.left);   makeRed(middle.right);
        }
        else {                                      // XXr → overflow
            makeBlack(parent);
            makeBlack(uncle);
            grand = parent.parent;
            if (grand != root) {    makeRed(grand);  resolveRed(grand);   }
        }
}  }
```